

SensorTrustFusion

Manual Técnico del Programa

Plataforma científica para fusión multisensor adaptativa bajo degradación e incertidumbre de sensores

Autor: Francisco Javier Burgos Flórez

Versión: 1.0.0

Fecha: Septiembre de 2026

Contacto: fjburgosf@gmail.com

Titular de los derechos: Francisco Javier Burgos Flórez

Repositorio: <https://github.com/fjburgosf/SensorTrustFusion>

Software científico desarrollado y validado exclusivamente con simulación computacional y sensores virtuales; no requiere hardware externo.

Tabla de contenido

1. Introducción.....	6
1.1 Propósito del documento.....	6
1.2 Descripción general del software.....	6
1.3 Carácter científico.....	6
2. Objetivo.....	7
2.1 Objetivo general.....	7
2.2 Objetivos específicos.....	7
2.3 Preguntas científicas que permite estudiar.....	7
3. Alcance.....	7
4. Fundamentos científicos.....	8
4.1 Problema de fusión bajo degradación.....	8
4.2 La innovación como evidencia.....	8
4.3 Modularidad de la formulación de confianza.....	8
5. Modelos dinámicos.....	8
5.1 Modelo general.....	8
5.2 Modelos predefinidos.....	9
5.3 Generador del valor verdadero.....	10
6. Modelo de sensores.....	10
6.1 Especificación.....	10
6.2 Cadena de medición.....	10
6.3 Covarianza nominal.....	11
7. Modelos de ruido.....	11
8. Motor de degradación.....	11
8.1 Perfiles temporales.....	11
8.2 Tipos de degradación.....	12
8.3 Valor verdadero de los fallos.....	13
8.4 Flujos aleatorios independientes.....	13
9. Innovación.....	13

10. Primer momento de la innovación.....	14
10.1 Referencia de consenso.....	14
11. Segundo momento de la innovación.....	15
12. Confiabilidad (SensorTrust).....	15
12.1 Interfaz común.....	15
12.2 Estimador de los dos primeros momentos.....	16
12.3 Compuerta NIS (referencia clásica).....	17
12.4 Detección de degradación.....	17
13. Algoritmos de fusión.....	17
13.1 Métodos de referencia estáticos.....	17
13.2 Fusión centralizada.....	17
13.3 Pesos.....	18
13.4 Fusión a nivel de estimaciones.....	18
13.5 Catálogo de métodos.....	18
14. Filtro de Kalman, EKF y UKF.....	19
14.1 Filtro de Kalman.....	19
14.2 Filtro de Kalman extendido.....	19
14.3 Filtro de Kalman unscented.....	19
14.4 Validación.....	19
15. Adaptación de covarianza.....	19
16. Monte Carlo y diseño de experimentos.....	20
16.1 Motor Monte Carlo.....	20
16.2 Diseños y distribuciones.....	20
16.3 Barridos paramétricos.....	20
16.4 Envolvente de robustez.....	21
17. Análisis de sensibilidad.....	21
18. Métricas.....	21
18.1 Estimación.....	21
18.2 Consistencia.....	22

18.3 Detección.....	22
18.4 Confianza y pesos.....	22
18.5 Robustez.....	22
18.6 Estadística.....	22
19. Arquitectura.....	23
19.1 Arquitectura científica.....	23
19.2 Capas.....	23
19.3 Idiomas, mensajes y manejo de errores.....	23
19.4 API interna.....	23
20. Módulos.....	24
21. Flujo de datos.....	24
22. Configuración.....	25
23. Formatos de datos.....	25
24. Reproducibilidad.....	26
25. Verificación del software.....	26
25.1 Robustez de la interfaz gráfica.....	27
26. Validación científica y resultados de referencia.....	27
26.1 Caso nominal y consistencia.....	28
26.2 Benchmarks de degradación.....	28
26.3 Otros modelos.....	31
26.4 Sensor dominante (ST-BENCH-16).....	32
26.5 Comparación de métodos (sesgo + ruido).....	32
26.6 Monte Carlo.....	33
26.7 Envolvente de robustez.....	35
26.8 Barrido paramétrico.....	36
26.9 Sensibilidad.....	36
26.10 Revisión final.....	37
27. Instalación.....	38
28. Ejecución.....	38

29. Limitaciones..... 39

Glosario..... 39

Referencias..... 39

1. Introducción

1.1 Propósito del documento

Este manual reúne la información técnica de **SensorTrustFusion v1.0.0**, *Plataforma científica para fusión multisensor adaptativa bajo degradación e incertidumbre de sensores*. Explica los fundamentos científicos en los que se apoya el programa, los modelos matemáticos y los algoritmos que implementa, cómo está organizado el código, qué archivos produce y cómo se garantiza que sus resultados sean reproducibles. Al final se presentan la verificación del software y los resultados de validación.

Las ecuaciones que aparecen aquí son las que el código implementa, y cuando una de ellas corresponde a un módulo concreto se indica en qué archivo está (`src/sensortrust/...`). Las cifras, tablas y figuras de resultados salen de ejecuciones reales del programa con las configuraciones incluidas en `benchmarks/` y `configs/`, todas con semilla fija, así que cualquiera puede reproducirlas con los comandos de la sección 26.

1.2 Descripción general del software

SensorTrustFusion sirve para diseñar y ejecutar experimentos de fusión multisensor cuando los sensores se degradan, fallan o están sujetos a incertidumbre. El programa simula la evolución verdadera de un sistema dinámico y las mediciones de tantos sensores virtuales como se necesiten, cada uno con su modelo de ruido y sus degradaciones. Sobre exactamente las mismas mediciones ejecuta distintos algoritmos de estimación y fusión, estima en cada instante la confiabilidad de cada sensor a partir de la innovación y mide el desempeño con métricas objetivas. También permite hacer estudios Monte Carlo, barridos de parámetros, envolventes de robustez y análisis de sensibilidad, todos reproducibles gracias a las semillas.

La versión 1.0.0 funciona por sí sola y **no necesita hardware externo**: ni microcontroladores, ni tarjetas de adquisición, ni sensores físicos, puertos seriales, buses de campo o instrumentos. Todos los experimentos, benchmarks, validaciones y capturas de pantalla de este manual se hicieron únicamente con simulación y sensores virtuales. Si se quiere, también es posible importar datos externos en CSV, TXT o JSON.

1.3 Carácter científico

Lo que le da carácter científico al software no es la cantidad de ecuaciones que contiene, sino el tipo de trabajo que permite hacer. Con él se diseñan experimentos controlados en los que se conoce el valor verdadero del sistema, se modelan sensores y degradaciones con perfiles temporales y se incorporan algoritmos nuevos. Se pueden formular hipótesis y ponerlas a prueba (por ejemplo, «el segundo momento de la innovación aporta información que no está en la media»), repetir un experimento muchas veces con Monte Carlo y medir la incertidumbre de los resultados con intervalos de confianza y pruebas pareadas. Los métodos se comparan siempre sobre los mismos datos; se cuantifican el desempeño, la robustez y la sensibilidad, y cualquier experimento anterior puede repetirse exactamente gracias a las semillas, los manifiestos y las huellas criptográficas. De paso, el programa produce conjuntos de datos que pueden reutilizarse en otros trabajos.

2. Objetivo

2.1 Objetivo general

Desarrollar una plataforma computacional reproducible para modelar, simular, analizar y comparar algoritmos de fusión multisensor en condiciones nominales y bajo incertidumbre, fallos y degradación no estacionaria, que permita estimar en línea la confiabilidad de las mediciones y cuantificar su efecto sobre la calidad de la estimación.

2.2 Objetivos específicos

1. Generar trayectorias verdaderas de sistemas dinámicos lineales y no lineales y mediciones de sensores virtuales con ruido, degradación y pérdida de datos configurables, conservando el valor verdadero de los fallos.
2. Implementar estimadores (KF, EKF, UKF), estrategias de adaptación de covarianza y un estimador de confianza basado en los dos primeros momentos de la innovación, desacoplados entre sí.
3. Comparar métodos de manera justa: todos los algoritmos se ejecutan sobre el mismo conjunto de datos de escenario.
4. Cuantificar error, consistencia, detección, confianza, pesos y robustez con métricas explícitas.
5. Ejecutar Monte Carlo, diseños de experimentos, barridos, envolventes de robustez y análisis de sensibilidad.
6. Garantizar la reproducibilidad completa de cada resultado.

2.3 Preguntas científicas que permite estudiar

El software se construyó para responder preguntas como estas: ¿cómo cambia la estimación cuando uno o varios sensores se degradan?, ¿cómo reacciona un algoritmo ante un sesgo, un aumento de la varianza o una deriva?, ¿se puede inferir la pérdida de confiabilidad a partir de la innovación?, ¿basta con la media de la innovación o el segundo momento aporta algo más? También interesa saber cuánto tarda cada algoritmo en reaccionar y en recuperar la confianza, qué pasa cuando fallan dos sensores a la vez, qué método mantiene el menor error con menos falsas alarmas, qué severidad se puede tolerar antes de incumplir una especificación y qué parámetros explican la variabilidad del desempeño. El capítulo 26 muestra resultados reales para varias de estas preguntas.

3. Alcance

La versión 1.0.0 incluye seis modelos dinámicos predefinidos y la posibilidad de definir modelos lineales propios. Los sensores virtuales pueden ser tantos como se quiera, cada uno con su frecuencia, resolución, rango, sesgo nominal, retardo y pérdida de datos. Hay ocho modelos de ruido, quince tipos de degradación y ocho perfiles temporales, uno de ellos de recuperación. Para la estimación trae filtros KF, EKF y UKF, fusión centralizada y a nivel de estimaciones, cuatro estrategias para la covarianza (nominal, verdadera u oráculo, adaptativa de Sage–Husa y SensorTrust), estimadores de confianza y detectores de degradación.

Para evaluar los resultados calcula métricas de estimación, de consistencia (NIS y NEES), de detección, de confianza, de pesos y de robustez. Los estudios se apoyan en un motor Monte Carlo que admite diseños aleatorios, por hipercubo latino, de Sobol o en rejilla, y se complementan con barridos paramétricos, envolventes y mapas de robustez, análisis de sensibilidad OAT, de Morris y de Sobol, estadística descriptiva y pruebas pareadas. El programa

se usa desde una interfaz gráfica bilingüe (español e inglés), desde la línea de comandos o desde Python; exporta a CSV, JSON, YAML, PNG, SVG y PDF, y puede importar datos externos.

Quedan fuera de esta versión la comunicación con hardware, la adquisición en tiempo real, las arquitecturas distribuidas con comunicación entre nodos, los sensores con dinámica propia (por ejemplo, con filtros internos) y la estimación de los parámetros del modelo dinámico.

4. Fundamentos científicos

4.1 Problema de fusión bajo degradación

Se consideran N_s sensores que observan el estado $x_k \in R^n$ de un sistema dinámico. Cada sensor i produce $z_{i,k} \in R^{m_i}$. Bajo la hipótesis nominal H_0 el sensor se comporta como especifica su hoja de datos: $z_{i,k} = h_i(x_k) + v_{i,k}$ con $v_{i,k} \sim N(0, R_i)$. Una degradación viola H_0 de forma no estacionaria (sesgo que aparece o crece, varianza creciente, deriva, congelamiento, valores atípicos, pérdida de datos, etc.). El estimador no conoce el instante, el tipo ni la magnitud de la degradación; debe inferirla a partir de los datos y ajustar la contribución de cada sensor.

4.2 La innovación como evidencia

En un filtro de Kalman consistente [1], [2] la innovación $v_{i,k} = z_{i,k} - \hat{z}_{i,k|k-1}$ es blanca, de media cero y covarianza $S_{i,k}$. Por lo tanto su versión estandarizada $\varepsilon_{i,k} = L_{i,k}^{-1} v_{i,k}$ ($S_{i,k} = L_{i,k} L_{i,k}^T$) satisface bajo H_0 :

$$E[\varepsilon_{i,k}] = 0, E[\varepsilon_{i,k} \varepsilon_{i,k}^T] = I_{m_i}. \quad (1)$$

Un sesgo desplaza el **primer momento**; un aumento de varianza incrementa el **segundo momento** sin alterar la media; un sesgo incrementa también el segundo momento, pero no la varianza central. SensorTrustFusion explota esta diferencia para estimar la confianza de cada sensor (capítulos 9 a 12).

4.3 Modularidad de la formulación de confianza

La formulación del estimador FirstTwoMomentsTrustEstimator se describe completa en el capítulo 12. El estimador está aislado en `src/sensortrust/trust/estimators.py` y la función `raw_trust`, que transforma los excesos de los momentos en confianza, puede sustituirse por formulaciones alternativas (heredando de la clase) sin modificar filtros, modelos, simulación ni interfaz. Esto permite usar la plataforma para comparar distintas funciones de confianza sobre los mismos escenarios.

5. Modelos dinámicos

5.1 Modelo general

El sistema verdadero evoluciona como

$$x_{k+1} = f(x_k, u_k) + w_k, w_k \sim N(0, Q), \quad (2)$$

y, en el caso lineal, $f(x_k, u_k) = Ax_k + Bu_k$. El periodo de muestreo es $T_s = 1/f_s$ y la rejilla temporal es $t_k = kT_s$, $k = 0, \dots, K-1$. Los modelos implementan la interfaz `DynamicModel` (`models/base.py`) con f , su jacobiano $F = \partial f / \partial x$ (analítico en modelos lineales, diferencias centrales en los no lineales), Q , las entradas deterministas u_k , los nombres y unidades de los estados y funciones de medición con nombre.

Los modelos continuos se discretizan exactamente con retenedor de orden cero y el método de Van Loan [3]:

$$A = e^{A_c T_s}, B = \int_0^{T_s} e^{A_c s} ds B_c, Q = \int_0^{T_s} e^{A_c s} Q_c e^{A_c^T s} ds. \quad (3)$$

5.2 Modelos predefinidos

Modelo 1 – Señal escalar variable (`scalar_signal`): $x_{k+1} = x_k + T_s u_k + w_k$, con $u(t) = 2\pi f A \cos(2\pi f t)$ y $Q = q T_s$; sin ruido de proceso, $x(t) = x_0 + A \sin(2\pi f t)$.

Modelo 2 – Velocidad constante (`constant_velocity`), $x = \dot{x}$:

$$A = \begin{bmatrix} 1 & T_s \\ 0 & 1 \end{bmatrix}, Q = q \begin{bmatrix} T_s^3/3 & T_s^2/2 \\ T_s^2/2 & T_s \end{bmatrix}. \quad (4)$$

Modelo 3 – Aceleración constante (`constant_acceleration`), $x = \ddot{x}$:

$$A = \begin{bmatrix} 1 & T_s & T_s^2/2 \\ 0 & 1 & T_s \\ 0 & 0 & 1 \end{bmatrix}, Q = q \begin{bmatrix} T_s^5/20 & T_s^4/8 & T_s^3/6 \\ T_s^4/8 & T_s^3/3 & T_s^2/2 \\ T_s^3/6 & T_s^2/2 & T_s \end{bmatrix}. \quad (5)$$

Modelo 4 – Oscilador masa-resorte-amortiguador (`mass_spring_damper`): $m \ddot{p} + c \dot{p} + k p = F(t) + w(t)$, con

$$A_c = \begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix}, B_c = \begin{bmatrix} 0 \\ 1/m \end{bmatrix}, Q_c = \begin{bmatrix} 0 & 0 \\ 0 & q/m^2 \end{bmatrix}, \quad (6)$$

discretizado con Van Loan y excitado por $F(t) = F_0 \sin(2\pi f t)$.

Modelo 5 – Sistema térmico simplificado (`thermal`): nodo calefactor T_h y nodo objeto T_o ,

$$C_h \dot{T}_h = P(t) - \frac{T_h - T_o}{R_{ho}}, C_o \dot{T}_o = \frac{T_h - T_o}{R_{ho}} - \frac{T_o - T_{amb}}{R_{oa}}, \quad (7)$$

con entradas $u = \dot{P}$ (potencia en onda cuadrada) y discretización exacta.

Modelo 6 – Péndulo no lineal (pendulum), $x=\dot{\zeta}$:

$$\dot{\theta}=\omega, \dot{\omega}=-\frac{g}{L}\sin\theta-b\omega+\tau(t)+w(t), \quad (8)$$

propagado con un paso de Runge–Kutta de cuarto orden por periodo de muestreo. Publica las funciones de medición `angle` (θ), `angular_rate` (ω), `horizontal_position` ($L\sin\theta$, no lineal) y `vertical_position` ($-L\cos\theta$, no lineal), que justifican el uso de EKF y UKF.

Modelo lineal personalizado (`custom_linear`): el usuario proporciona A , B , Q discretas o A_c , B_c , Q_c continuas (con `continuous: true`), nombres y unidades de estados y señales de entrada. Nuevos modelos en Python se registran con el decorador `register_model`.

5.3 Generador del valor verdadero

`GroundTruthSimulator` (`simulation/ground_truth.py`) genera $x_0 \sim N(\dot{x}_0, P_0)$ (o $x_0 = \dot{x}_0$) y propaga $x_{k+1} = f(x_k, u_k) + w_k$ con $w_k = L_Q \eta_k$, $L_Q L_Q^T = Q$ (Cholesky o descomposición espectral si Q es semidefinida). El valor verdadero x_k^{true} se conserva para todo el horizonte y solo lo utiliza el entorno experimental (generación de sensores y evaluación). Los estimadores reciben únicamente el objeto `MeasurementSet` (tiempos, mediciones, entradas conocidas, especificaciones nominales de los sensores y la distribución a priori del estado inicial); los métodos marcados como **oráculo** reciben explícitamente `OracleInfo` y se etiquetan como referencias no realizables.

6. Modelo de sensores

6.1 Especificación

Cada sensor virtual (`sensors/sensor.py`) se configura con: nombre; variable medida (índice o nombre del estado, matriz H_i explícita o función de medición con nombre); frecuencia; modelo de ruido; desviación supuesta por los estimadores; sesgo nominal no compensado; resolución; rango; retardo de transporte; probabilidad nominal de pérdida; confiabilidad nominal $\rho_i^0 \in (0, 1)$ y lista de degradaciones. No existe límite en el número de sensores.

6.2 Cadena de medición

Con retardo de d muestras, la medición del sensor i en el paso base k se construye como:

$$c_k = h_i(x_{k-d}), y_k = \alpha_k (c_k + b_i^{nom} + b_k^{deg} + v_k + v_k^{deg}), \quad (9)$$

$$z_k = stuck(post^{deg}(Q_\Delta(sat_{[l,u]}(y_k)))) \quad (10)$$

donde α_k es el producto de los factores multiplicativos (pérdida de sensibilidad), b_i^{nom} el sesgo nominal, b_k^{deg} la suma de las degradaciones aditivas, v_k el ruido nominal, v_k^{deg} el ruido de degradación, sat el rango nominal, $Q_\Delta(y) = \text{Round}(y/\Delta)$ la resolución nominal, $post^{deg}$ las degradaciones de posprocesamiento (saturación y cuantización degradadas) y `stuck` el congelamiento. Finalmente se aplica la máscara de disponibilidad: el sensor solo produce muestra

cuando $k \bmod round(f_s/f_i) = 0$ y la muestra no se pierde por pérdida nominal ni por degradación; las muestras no disponibles se representan como NaN.

El valor verdadero asociado a cada sensor incluye la medición limpia c_k , el error sistemático verdadero y la varianza verdadera del ruido:

$$b_k^{true} = \alpha_k (c_k + b_i^{nom} + b_k^{deg}) - c_k, R_k^{true} = \alpha_k^2 (Var[v_k] + Var[v_k^{deg}]). \quad (11)$$

6.3 Covarianza nominal

Los estimadores usan $R_i = diag(\sigma_i^2)$, donde σ_i es la desviación nominal del modelo de ruido o, si se indica, la desviación supuesta (assumed_std), lo que permite estudiar desajustes de modelo.

7. Modelos de ruido

sensors/noise.py implementa ocho modelos; cada uno devuelve la realización v_k y su varianza verdadera instantánea:

Tabla 1. Modelos de ruido implementados.

Tipo	Definición	Varianza verdadera
Gaussiano (gaussian)	$v_k \sim N(0, \sigma^2)$	σ^2
Uniforme (uniform)	$v_k \sim U(-a, a), a = \sqrt{3}\sigma$	σ^2
Variante en el tiempo (time_varying)	$\sigma_k = \sigma(1 + A \sin(2\pi t_k/P))$	σ_k^2
Dependiente del tiempo (time_dependent)	$\sigma_k = \sigma + s t_k$	σ_k^2
Heterocedástico (heteroscedastic)	$\sigma_k = \sigma + g \vee h(x_k) \vee i$	σ_k^2
Correlacionado AR(1) (correlated)	$v_k = \phi v_{k-1} + \sqrt{1 - \phi^2} \sigma \eta_k, \phi = e^{-T_s/\tau_c}$	σ^2
En ráfagas (burst)	σ multiplicada por F durante ráfagas markovianas	σ^2 o $F^2 \sigma^2$
Impulsivo (impulsive)	gaussiano más impulsos $N(0, \sigma_{imp}^2)$ con probabilidad p	$\sigma^2 + p \sigma_{imp}^2$

En el ruido en ráfagas, las probabilidades de transición por paso son $p_{on} = 1 - e^{-\lambda_b T_s}$ y $p_{off} = 1 - e^{-T_s/D_b}$, con λ_b la tasa de ráfagas y D_b su duración media.

8. Motor de degradación

8.1 Perfiles temporales

Cada degradación es modulada por una activación $g(t) \in [0, 1]$ (degradation/profiles.py):

$$g(t) = \begin{cases} 0, & t < t_f \\ o(t - t_f)r(t), & t \geq t_f \end{cases} \quad (12)$$

con forma de inicio $o(\tau)$ y envolvente de recuperación $r(t)$ (T_r : tiempo de subida):

Tabla 2. Perfiles temporales de degradación.

Perfil	Forma de inicio $o(\tau)$
Escalón (step)	1
Rampa (ramp)	$\min(1, \tau/T_r)$
Exponencial (exponential)	$1 - e^{-\tau/T_r}$
Sigmoide (sigmoid)	$\frac{s(\tau) - s(0)}{s(T_r) - s(0)}, s(\tau) = (1 + e^{-10(\tau/T_r - 1/2)})^{-1}; 1 \text{ si } \tau \geq T_r$
Sinusoidal (sinusoidal)	$(1 - \cos(2\pi\tau/P))/2$
Caminata aleatoria (random_walk)	$o_k = \text{clip}(o_{k-1} + \sigma_{rw} \sqrt{T_s} \eta_k, 0, 1)$
Intermitente (intermittent)	cadena de Markov encendido/apagado con duraciones medias dadas
Recuperación (recovery)	escalón (o rampa) que exige instante de recuperación

La envolvente de recuperación, con instante t_r y duración D_r , es $r(t) = 1$ para $t < t_r$ y, para $t \geq t_r$, lineal $r = \max(0, 1 - (t - t_r)/D_r)$ (con $D_r = 0$ el fallo termina abruptamente) o exponencial $r = e^{-(t - t_r)/D_r}$. El parámetro *duration* equivale a $t_r = t_f + \text{duration}$. De esta forma el usuario decide cuándo empieza el fallo, cuánto dura, qué tan severo es, a qué velocidad se desarrolla y cuándo y con qué rapidez se recupera el sensor.

8.2 Tipos de degradación

Con severidad s , tiempo desde el inicio $\tau_k = t_k - t_f$ y periodo base T_s (degradation/faults.py):

Tabla 3. Tipos de degradación implementados.

Tipo	Etapas	Efecto
Sesgo constante (bias)	aditiva	$b_k = s g_k$
Sesgo progresivo (progressive_bias)	aditiva	$b_k = (b_0 + s \tau_k) r_k$ (s = tasa)
Deriva (drift)	aditiva	$d_k = d_{k-1} + s T_s g_k$ (acumulativa, persiste tras la recuperación)
Deriva de caminata aleatoria (random_walk_drift)	aditiva	$d_k = d_{k-1} + s \sqrt{T_s} \eta_k g_k$
Periódica (periodic)	aditiva	$b_k = s \sin(2\pi\tau_k/P) g_k$
Fallo intermitente (intermittent)	aditiva	$b_k = s c_k g_k$, c_k cadena de Markov
Varianza creciente (variance_increase)	ruido	$R_k = R_0 (1 + (s - 1) g_k)$ (s = factor final)
Fallo en ráfagas (burst_fault)	ruido	ráfagas de Poisson de ruido $N(0, s^2)$
Valores atípicos (outliers)	ruido	$\pm s(0, 5 + U(0, 1))$ con probabilidad $p g_k$
Pérdida de sensibilidad (sensitivity_loss)	multiplicativa	$\alpha_k = 1 - s g_k$
Saturación (saturation)	posproceso	$\text{clip}(z, l, u)$ mientras $g_k > 0$

Tipo	Etap	Efecto
Cuantización (quantization)	posproceso	$sround(z/s)$ mientras $g_k > 0$
Sensor congelado (stuck)	posproceso	$z_k = z_{k_f}$ mientras $g_k \geq 1/2$
Pérdida de muestras (dropout)	disponibilidad	muestra perdida con probabilidad sg_k
Pérdida de paquetes (packet_loss)	disponibilidad	pérdidas en ráfaga de Gilbert–Elliott, $P(\text{bueno} \rightarrow \text{malo}) = s$, longitud media L

Con el perfil rampa, `progressive_bias` y `variance_increase` reproducen exactamente $b_k = b_0 + \alpha(k - k_f)$ y $R_k = R_0 + \alpha(k - k_f)$. La degradación `combined` agrupa varias componentes con un perfil común (por ejemplo, sesgo más ruido creciente).

8.3 Valor verdadero de los fallos

Para cada sensor se guarda, muestra a muestra, si está en estado nominal o degradado ($F_i(k) = 1$ cuando alguna degradación está activa, es decir, $g_k > 0,01$, o cuando un fallo intermitente está «encendido»). También se guardan el instante de inicio configurado, el primer y el último instante en que el fallo estuvo activo, su tipo, severidad y perfil, el instante y la duración de la recuperación, y los valores verdaderos b_k^{true} y R_k^{true} . Con esta información, la detección, la confianza y los pesos se evalúan de forma objetiva.

8.4 Flujos aleatorios independientes

Cada proceso estocástico usa su propio generador derivado de la semilla y de una clave (`utils/rng.py`): ("truth", "x0"), ("truth", "process"), ("sensor", i, "noise"), ("sensor", i, "dropout"), ("sensor", i, "degradation", j), ("sensor", i, "profile", j). En consecuencia, eliminar una degradación no altera las realizaciones de ruido: el **gemelo nominal** de un escenario comparte exactamente el mismo ruido y valor verdadero, lo que habilita comparaciones pareadas nominal vs. degradado.

9. Innovación

Para los métodos basados en filtros (`fusion/centralized.py`), en cada paso y para cada sensor con muestra disponible se calcula en el estado a priori:

$$v_{i,k} = z_{i,k} - \hat{z}_{i,k \vee k-1}, S_{i,k} = H_{i,k} P_{k \vee k-1} H_{i,k}^T + R_i, \varepsilon_{i,k} = L_{i,k}^{-1} v_{i,k}, \quad (13)$$

con $\hat{z}_{i,k \vee k-1} = h_i(\hat{x}_{k \vee k-1})$ en KF/EKF (o la media de los puntos sigma en UKF) y $L_{i,k}$ el factor de Cholesky de $S_{i,k}$. Se utiliza la covarianza **nominal** R_i y no la efectiva: el estadístico contrasta así la hipótesis «el sensor se comporta como está especificado»; con la covarianza efectiva, un sensor ya penalizado tendría innovaciones estandarizadas artificialmente pequeñas y recuperaría la confianza de forma espuria.

Se almacenan las series completas de $v_{i,k}$, $\sqrt{\text{diag} S_{i,k}}$, $\varepsilon_{i,k}$, el NIS por sensor $\varepsilon_{i,k}^T \varepsilon_{i,k}$ y el NIS total de la actualización conjunta con la covarianza efectiva:

$$NIS_k = v_k^T S_k^{-1} v_k, v_k = \textcolor{red}{\hat{z}_k} \quad (14)$$

donde A_k es el conjunto de sensores con muestra. Los métodos estáticos (promedios) no tienen predictor y, por lo tanto, no reportan innovación ni NIS.

10. Primer momento de la innovación

El primer momento $\mu_{v_i} = E[\varepsilon_i]$ se estima en línea (`trust/moments.py`) con tres estimadores:

EWMA:

$$\hat{\mu}_k = \lambda \hat{\mu}_{k-1} + (1 - \lambda) \varepsilon_k, N_{eff} = \frac{1 + \lambda}{1 - \lambda}, \quad (15)$$

inicializado en el valor nominal $\hat{\mu}_0 = 0$.

Ventana deslizante:

$$\hat{\mu}_k = \frac{1}{N_k} \sum_{j=k-N_k+1}^k \varepsilon_j, N_k = \min(n_k, N), N_{eff} = N_k. \quad (16)$$

Media recursiva:

$$\hat{\mu}_k = \hat{\mu}_{k-1} + \frac{\varepsilon_k - \hat{\mu}_{k-1}}{n_k}, N_{eff} = n_k. \quad (17)$$

n_k es el número de muestras disponibles recibidas. Bajo H_0 la desviación estándar de cada componente de $\hat{\mu}$ es $1/\sqrt{N_{eff}}$. Para sensores escalares se almacena $\hat{\mu}$ con signo; para sensores vectoriales, $\|\hat{\mu}\|/\sqrt{m}$.

10.1 Referencia de consenso

La innovación se calcula respecto a la predicción fusionada, común a todos los sensores. Si el sensor degradado es el más preciso y domina la estimación, la estimación lo sigue y son los sensores **sanos** los que muestran las medias de innovación más grandes. Sin embargo, un error común de estimación e desplaza la media de innovación de todos los sensores de un **grupo redundante** G (sensores con la misma función de medición) en la misma cantidad $-He$, mientras que un fallo desplaza solo la media del sensor afectado. SensorTrustFusion referencia por ello las medias físicas $p_i = L_i \hat{\mu}_i$ a la mediana de los miembros confiables del grupo:

$$\hat{\mu}_i^c = L_i^{-1} \left(L_i \hat{\mu}_i - \underset{j \in G_T}{\text{median}} L_j \hat{\mu}_j \right), G_T = \{ j \in G : T_j \geq f_c \max_{l \in G} T_l \}, \quad (18)$$

aplicada cuando el grupo tiene al menos `consensus_min_sensors` (≥ 3) sensores con muestra. Si hay menos de dos miembros confiables, la mediana se toma sobre los miembros que fueron

confiables dentro de la ventana de memoria `consensus_memory_samples` (200 muestras), siempre que sean al menos `consensus_min_sensors`; en otro caso no se corrige. Así se evita un bloqueo en el que el ruido reduce momentáneamente la confianza de los sensores sanos en el instante del fallo y el sensor defectuoso queda como única referencia (caso identificado con el análisis de sensibilidad de Sobol, sección 26.9), mientras que los sensores desconfiados de forma persistente permanecen fuera de la referencia (fallos secuenciales o mayoritarios, ST-BENCH-12). La mediana es robusta ante una minoría de sensores defectuosos y la restricción a miembros confiables evita que sensores ya aislados contaminen la referencia de fallos posteriores. En grupos de dos sensores no se corrige: un desacuerdo entre dos sensores no puede atribuirse sin redundancia adicional. El benchmark ST-BENCH-16 (sección 26.4) documenta el efecto de esta referencia.

11. Segundo momento de la innovación

El segundo momento normalizado por dimensión se estima con el mismo tipo de estimador que el primero, aplicado a $q_k = \varepsilon_k^T \varepsilon_k / m$:

$$\hat{m}_{2,k} = \lambda \hat{m}_{2,k-1} + (1 - \lambda) \frac{\varepsilon_k^T \varepsilon_k}{m} \quad (\hat{m}_{2,0} = 1 \text{ en EWMA}), \quad (19)$$

o su equivalente en ventana deslizante o media recursiva. Bajo H_0 , $E[\hat{m}_2] = 1$. El software distingue explícitamente cuatro cantidades que solo coinciden para innovaciones de media cero:

Tabla 4. Cantidades estadísticas asociadas al segundo momento de la innovación.

Cantidad	Definición implementada
Segundo momento	$\hat{m}_2 \approx E[\ \varepsilon\ ^2] / m$
Varianza (momento central)	$\hat{s}^2 = \max(0, \hat{m}_2 - \ \hat{\mu}\ ^2 / m)$
RMS	$\sqrt{\hat{m}_2}$
Energía (en la ventana)	$\sum_j \ \varepsilon_j\ ^2 = N m \hat{m}_2$

Consecuencia observable (figura del capítulo 26): un sesgo eleva $\hat{m}_2$ pero no $\hat{s}^2$, mientras que un aumento de ruido eleva ambos; por ello el segundo momento, a través de la varianza central, aporta información que la media no contiene. La varianza así estimada es el estimador sesgado (divisor N) coherente con los momentos.

12. Confiabilidad (SensorTrust)

12.1 Interfaz común

Todos los estimadores de confianza implementan `TrustEstimator` (`trust/estimators.py`): `reset(specs, K)` y `update(k, eps, S) -> T`, donde `eps[i]` es la innovación estandarizada (o `None` si no hay muestra) y `S[i]` su covarianza nominal. Disponibles: `first_two_moments` (método principal), `first_moment_only` y `second_moment_only` (ablaciones), `nis_gate` (compuerta chi-cuadrado clásica) y `constant` (sin adaptación).

12.2 Estimador de los dos primeros momentos

Con $\hat{\mu}_i^c$, $\hat{s}_i^2$ y N_{eff} :

$$d_{1,i}(k) = \max\left(0, \frac{\|\hat{\mu}_i^c(k)\|}{\sqrt{m}} - \frac{\gamma_1}{\sqrt{N_{eff}}}\right), d_{2,i}(k) = \max\left(0, \hat{s}_i^2(k) - 1 - \gamma_2 \sqrt{\frac{2}{N_{eff}}}\right), \quad (20)$$

$$\tilde{T}_i(k) = \rho_i^0 \exp\left(-\frac{d_{1,i}(k)}{\beta_1} - \frac{d_{2,i}(k)}{\beta_2}\right). \quad (21)$$

$\gamma_1/\sqrt{N_{eff}}$ y $\gamma_2\sqrt{2/N_{eff}}$ son zonas muertas estadísticas (γ desviaciones estándar de los estimadores bajo H_0); β_1 es el exceso de media (en desviaciones de la innovación) que divide la confianza por e y β_2 el exceso análogo de varianza; ρ_i^0 es la confiabilidad nominal. Con dispersion: second_moment se usa $\hat{m}_2$ en lugar de $\hat{s}^2$ en d_2 .

La dinámica de confianza es asimétrica (rápida al disminuir, lenta al recuperarse):

$$T_i(k) = \min\left(1, \max\left(T_{min}, T_i(k-1) + \eta[\tilde{T}_i(k) - T_i(k-1)]\right)\right), \quad (22)$$

$$\eta = \eta_i \text{ si } \tilde{T}_i(k) < T_i(k-1); \eta = \eta_\uparrow \text{ en otro caso.} \quad (23)$$

Si un sensor no produce muestra, sus momentos y su confianza se mantienen. Parámetros y valores por defecto:

Tabla 5. Parámetros del estimador de los dos primeros momentos y valores por defecto.

Símbolo	Clave de configuración	Valor por defecto
estimador de momentos	estimator	ewma
λ	ewma_lambda	0.98
N	window_length	100
γ_1	mean_deadzone	3.0
γ_2	dispersion_deadzone	3.0
β_1	mean_scale	0.5
β_2	dispersion_scale	1.0
η_i	trust_decrease_rate	0.5
$\eta_\uparrow$	trust_recovery_rate	0.02
T_{min}	trust_floor	0.01
uso de $\hat{\mu} / \hat{s}^2$	use_first_moment / use_second_moment	verdadero
referencia de consenso	consensus, consensus_min_sensors, consensus_trust_fraction (f_c), consensus_memory_samples	verdadero, 3, 0.5, 200

12.3 Compuerta NIS (referencia clásica)

$T_i(k) = \rho_i^0$ si $\varepsilon_{i,k}^T \varepsilon_{i,k} \leq \chi_{m,1-\alpha}^2$ y T_{min} en caso contrario (sin memoria).

12.4 Detección de degradación

La detección, $D_i(k) \in \{0, 1\}$, está separada conceptualmente de la ponderación (trust/detection.py): un método puede reducir la confianza sin declarar alarma y un detector no modifica pesos. El detector moment_test usa pruebas sobre los mismos momentos:

$$\text{prueba de media: } N_{eff} \|\hat{\mu}_i^c\|^2 > \chi_{m,1-\alpha}^2; \text{ prueba de dispersión: } v \hat{s}_i^2 > \chi_{v,1-\alpha}^2, v = m N_{eff}, \quad (24)$$

(la media de $\|\varepsilon\|^2/m$ se aproxima por una chi-cuadrado escalada, más precisa en la cola derecha que la aproximación gaussiana). D_i pasa a 1 tras alarm_on_samples alarmas crudas consecutivas y vuelve a 0 tras alarm_off_samples muestras consecutivas sin alarma (histéresis). El detector trust_threshold declara $D_i=1$ cuando T_i permanece bajo un umbral y lo libera sobre otro. Se almacenan el instante real del fallo y el instante de detección para calcular $T_{delay} = t_{detection} - t_{fault}$.

13. Algoritmos de fusión

Todos los métodos implementan FusionMethod.run(meas, model, oracle) -> MethodResult (fusion/). MethodResult contiene la estimación, P_k (si existe), innovaciones, NIS, confianza, pesos, alarmas, momentos, R^{eff} y $\hat{R}$.

13.1 Métodos de referencia estáticos

Válidos solo cuando todos los sensores miden directamente la misma componente x_j (en otro caso se reportan como «no aplicables»); estiman únicamente x_j y, si no hay muestras, mantienen el último valor:

$$\hat{x} = \frac{1}{N_k} \sum_{i \in A_k} z_i, \hat{x} = \frac{\sum_{i \in A_k} w_i z_i}{\sum_{i \in A_k} w_i}, w_i \propto \frac{1}{\sigma_i^2} \text{ (inverso de varianza)}. \quad (25)$$

El **mejor sensor (oráculo)** selecciona, con el valor verdadero y fuera de línea, el sensor de menor RMSE; es una referencia ideal no realizable.

13.2 Fusión centralizada

Las mediciones disponibles se apilan

$$z_k = \begin{bmatrix} z_{1,k} \\ \vdots \\ z_{N,k} \end{bmatrix}, H = \begin{bmatrix} H_1 \\ \vdots \\ H_N \end{bmatrix}, R_k^{eff} = \text{blkdiag}(R_{1,k}^{eff}, \dots, R_{N,k}^{eff}), \quad (26)$$

eliminando las muestras ausentes, y se procesan en una única actualización (KF, EKF o UKF). Un sensor puede removerse (use_sensors) o reponderarse mediante $R_{i,k}^{eff}$ según la estrategia de covarianza (capítulo 15).

13.3 Pesos

Para métodos con filtro se reporta el peso de información relativo de cada sensor disponible:

$$w_i(k) = \frac{1/r_i(k)}{\sum_{j \in A_k} 1/r_j(k)}, r_i(k) = \frac{tr R_{i,k}^{eff}}{m_i}, \quad (27)$$

de modo que $\sum_i w_i(k) = 1$. Para sensores que miden la misma cantidad son los pesos relativos de las mediciones en la actualización de Kalman y, en SensorTrust con varianzas nominales iguales, se reducen a $w_i = T_i / \sum_j T_j$. Las divisiones por cero se evitan con $T_{min} > 0$ y $r_i > 0$.

13.4 Fusión a nivel de estimaciones

Cada sensor alimenta su propio filtro local ($\hat{x}_i, P_i$). Como los errores locales están correlacionados de forma desconocida (ruido de proceso común), se fusiona con Intersección de Covarianzas [4]:

$$P_f^{-1} = \sum_i \omega_i P_i^{-1}, \hat{x}_f = P_f \sum_i \omega_i P_i^{-1} \hat{x}_i, \sum_i \omega_i = 1, \quad (28)$$

con $\omega_i = 1/N$ (ci_fusion) o $\omega_i = T_i / \sum_j T_j$ (sensortrust_ci). En la segunda, la innovación usada para la confianza se calcula respecto a la predicción fusionada $\hat{x}_f^- = f(\hat{x}_f^-)$, $P_f^- = F P_f F^T + Q$.

13.5 Catálogo de métodos

Tabla 6. Catálogo de métodos de estimación y fusión.

Clave	Descripción
simple_average, fixed_weights, inverse_variance	promedios estáticos
best_sensor_oracle	mejor sensor individual (oráculo)
kf	KF centralizado, R nominal (caso A)
kf_true_r	KF con R verdadera variable (oráculo, caso B)
adaptive_kf	KF con R estimada Sage–Husa (caso C)
sensortrust_kf	KF con ponderación SensorTrust (caso D)
sensortrust_kf_mean_only, sensortrust_kf_m2_only	ablaciones de momentos
kf_nis_gate	KF con compuerta NIS
single_sensor_kf	KF con subconjunto de sensores
ekf, adaptive_ekf, sensortrust_ekf	variantes EKF
ukf, sensortrust_ukf	variantes UKF
ci_fusion, sensortrust_ci	fusión a nivel de estimaciones

14. Filtro de Kalman, EKF y UKF

14.1 Filtro de Kalman

Implementado en `estimators/filters.py` (KalmanFilter), según la formulación clásica [1]:

$$\hat{x}_{k \vee k-1} = A \hat{x}_{k-1 \vee k-1} + B u_{k-1}, P_{k \vee k-1} = A P_{k-1 \vee k-1} A^T + Q, \quad (29)$$

$$v_k = z_k - H \hat{x}_{k \vee k-1}, S_k = H P_{k \vee k-1} H^T + R, K_k = P_{k \vee k-1} H^T S_k^{-1}, \quad (30)$$

$$\hat{x}_{k \vee k} = \hat{x}_{k \vee k-1} + K_k v_k, P_{k \vee k} = (I - K_k H) P_{k \vee k-1} (I - K_k H)^T + K_k R K_k^T. \quad (31)$$

Estabilidad numérica: la ganancia se obtiene resolviendo el sistema lineal $S_k K_k^T = H P_{k \vee k-1}^T$ (sin invertir S_k), la covarianza se actualiza en forma de Joseph y se simetriza en cada paso. Con x_0 a priori ($\hat{x}_0, P_0$) la primera actualización se realiza en $k=0$ y la predicción usa u_{k-1} , coherente con $x_k = f(x_{k-1}, u_{k-1}) + w_{k-1}$.

14.2 Filtro de Kalman extendido

Siguiendo [2], sustituye A y H por los jacobianos $F_k = \partial f / \partial x|_{\hat{x}_{k-1 \vee k-1}}$ y $H_k = \partial h / \partial x|_{\hat{x}_{k \vee k-1}}$, y propaga la media con f y h no lineales. Su uso se justifica con el péndulo no lineal y los sensores de posición $L \sin \theta$ (ST-BENCH-13).

14.3 Filtro de Kalman unscented

Transformación unscented escalada [5] con $\lambda = \alpha^2(n + \kappa) - n$, $2n+1$ puntos sigma $X_0 = \hat{x}$, $X_{\pm i} = \hat{x} \pm (\sqrt{(n + \lambda)P})_i$ y pesos $W_0^{(m)} = \lambda / (n + \lambda)$, $W_0^{(c)} = W_0^{(m)} + 1 - \alpha^2 + \beta$, $W_i = 1 / (2(n + \lambda))$. Por defecto $\alpha = 0,5$, $\beta = 2$, $\kappa = 0$. La actualización usa P_{zz} , P_{xz} de los puntos sigma, $K = P_{xz} S^{-1}$ y $P = P - K S K^T$.

14.4 Validación

Se verificó que la covarianza estacionaria coincide con la solución de la ecuación algebraica de Riccati discreta (`scipy.linalg.solve_discrete_are`), la varianza estacionaria analítica del caso escalar, error nulo con sensor sin ruido, consistencia (NIS/m ≈ 1 , NEES/n ≈ 1 , innovación blanca), dimensiones, simetría y definición positiva de P_k , e igualdad KF = EKF = UKF en modelos lineales.

15. Adaptación de covarianza

La opción `r_strategy` de la fusión centralizada define cómo se trata la covarianza de medición. Hay cuatro enfoques, y poder compararlos es uno de los propósitos del programa:

Caso A – R_i nominal fija: $R_{i,k}^{eff} = R_i$.

Caso B – R_i verdadera conocida (oráculo): $R_{i,k}^{eff} = \text{diag}(R_{i,k}^{true})$. Conoce la varianza real del ruido, **no** el sesgo.

Caso C – R_i estimada en línea (Sage–Husa [6], emparejamiento de covarianza [7]):

$$\hat{R}_{i,k} = (1 - d_k) \hat{R}_{i,k-1} + d_k (v_{i,k} v_{i,k}^T - H_i P_{k \vee k-1} H_i^T), d_k = \frac{1-b}{1-b^{k+1}}, \quad (32)$$

seguido de la proyección $\hat{R}_{i,k} \leftarrow R_{min} + \Pi_{\geq 0}(\hat{R}_{i,k} - R_{min})$ con $R_{min} = r_{min} R_i$ (recorte de valores propios), $b = \text{forgetting_factor}$ (0,99) y $r_{min} = \text{min_variance_ratio}$ (0,1).

Caso D – ponderación SensorTrust:

$$R_{i,k}^{eff} = \frac{R_i}{\max \left(T_{min}, T_i(k) / \max_{j \in A_k} T_j(k) \right)}. \quad (33)$$

Se usa la confianza relativa: el sensor más confiable conserva su R_i nominal, lo que evita que una reducción común de confianza (p. ej., al inicio) deje al filtro sin información.

Compuerta de innovación (caso D): una muestra con $\varepsilon_{i,k}^T \varepsilon_{i,k} > \chi_{m_i, 1-p_g}^2$ ($p_g = 10^{-6}$ por defecto) se excluye de la actualización del estado de ese paso (sigue alimentando los momentos). Evita que un error abrupto de muchas desviaciones en un sensor dominante arrastre la predicción antes de que los momentos reaccionen. Si todas las muestras disponibles superan la compuerta, no se excluye ninguna (se evita el bloqueo tras interrupciones largas).

16. Monte Carlo y diseño de experimentos

16.1 Motor Monte Carlo

Para cada corrida $r=1, \dots, N$ (`experiments/montecarlo.py`): (1) la semilla s_r se deriva de la semilla maestra con `numpy.random.SeedSequence`; (2) los parámetros variables se muestrean del diseño (semilla maestra); (3) se genera **un** escenario con (`configuración+parámetros, s_r`); (4) todos los métodos se ejecutan sobre ese mismo escenario y se evalúan. Cada fila de `runs.csv` conserva la corrida, la semilla, los parámetros y la huella del conjunto de datos, de modo que cualquier corrida puede re-ejecutarse de forma exacta. Las corridas se paralelizan con procesos (`concurrent.futures`); el resultado es idéntico al secuencial.

16.2 Diseños y distribuciones

Diseños (`experiments/design.py`): aleatorio (Monte Carlo simple), hipercubo latino [8] (`scipy.stats.qmc.LatinHypercube`), secuencia de Sobol aleatorizada (`scipy.stats.qmc.Sobol`) y rejilla factorial. Cada punto del hipercubo unitario se transforma con la inversa de la distribución del parámetro: uniforme, log-uniforme, normal, normal truncada, entera y elección discreta. El diseño generado se guarda con el estudio (`design.csv`).

16.3 Barridos paramétricos

Rejillas factoriales de uno o dos parámetros (`experiments/sweep.py`) con R repeticiones por punto. La repetición r usa la misma semilla s_r en todos los puntos (números aleatorios comunes), de modo que las diferencias entre puntos se deben a los parámetros. Se agregan media,

mediana, percentiles 5/95 y tasa de éxito, y se generan mapas como $R M S E(b, \sigma)$ y $Retardo(b, \sigma)$.

16.4 Envolvente de robustez

Para un parámetro de degradación b con valores ordenados $b_1 < \dots < b_j$ (experiments/robustness.py) y el estadístico $g(b_j)$ (media de la métrica o tasa de éxito sobre las repeticiones):

$$b^* = \{b_j : g(b_j) \text{ cumple la especificación } \forall b_i \leq b_j\}, \quad (34)$$

refinado por interpolación lineal de g entre el último valor que cumple y el primero que falla. Si todos los valores cumplen se informa $b^* \geq b_j$ («límite no alcanzado»); si el menor valor falla, no existe b^* . Con criterio de media, cumplir significa $\hat{g} \leq R M S E_{max}$; con criterio de tasa de éxito, $P(\text{métrica} \leq \text{espec.}) \geq p_{req}$. El mapa de robustez calcula la tasa de éxito en una rejilla de dos parámetros.

17. Análisis de sensibilidad

El modelo estudiado es $\theta \mapsto y = \text{métrica}(\text{método}, \text{escenario}(\theta, s))$ (experiments/sensitivity.py). Con seed_mode: common todas las evaluaciones usan el mismo escenario aleatorio, de modo que los índices miden el efecto de los parámetros y no del ruido.

Un factor a la vez (OAT): cada parámetro recorre levels valores en $[a, b]$ con los demás en su valor nominal; se reporta el rango de salida por parámetro.

Morris [9] (efectos elementales, SALib [10]): $E E_i = [y(\theta + \Delta e_i) - y(\theta)] / \Delta$ sobre N trayectorias en una rejilla de num_levels niveles ($N(D+1)$ evaluaciones); $\mu_i^* = E \vee E E_i \vee \sigma_i$ mide la importancia y σ_i la no linealidad o interacción.

Sobol (SALib, esquema de Saltelli [11]): índices de primer orden $S_i = Var[E(Y/\theta_i)] / Var(Y)$ y totales $S_{T_i} = E[Var(Y/\theta_{\sim i})] / Var(Y)$ con intervalos bootstrap, usando $N(D+2)$ evaluaciones.

18. Métricas

18.1 Estimación

Para la componente evaluada j (metrics.target_state) y el conjunto de evaluación I (todas las muestras tras el calentamiento, o una fase), con $e_k = \hat{x}_{k,j} - x_{k,j}$:

$$R M S E = \sqrt{\frac{1}{|I| \vee \sum_{k \in I} e_k^2}}, M S E = \frac{1}{|I| \vee \sum_{k \in I} e_k^2}, M A E = \frac{1}{|I| \vee \sum_{k \in I} |e_k|}, N R M S E = \frac{1}{\max_k} \quad (35)$$

además de error máximo $\max \vee e_k$, sesgo $\hat{e}$, varianza del error (divisor $n - 1$), error final e_K y RMSE antes, durante y después de los fallos (fases definidas por el valor verdadero de fallos de todos los sensores). También se reporta el RMSE de cada estado.

18.2 Consistencia

$NIS_k = v_k^T S_k^{-1} v_k \sim \chi^2(m_k)$ y $NEES_k = (x_k - \hat{x}_k)^T P_k^{-1} (x_k - \hat{x}_k) \sim \chi^2(n)$ para un filtro consistente; se reportan el valor medio normalizado ($\overline{NIS}/m$, $\overline{NEES}/n$, ≈ 1) y la fracción de pasos dentro del intervalo chi-cuadrado bilateral del 95 %. NIS solo se calcula para métodos con innovación y covarianza; NEES solo para métodos con P_k y estimación del estado completo.

18.3 Detección

Solo para métodos que declaran alarmas, por muestra y tras el calentamiento: VP, FP, FN, VN; precisión $VP/(VP+FP)$; exhaustividad $VP/(VP+FN)$; $F_1 = 2PR/(P+R)$; tasa de falsa alarma $FP/(FP+VN)$; tasa de detección fallida $FN/(VP+FN)$; retardo $T_{delay} = t_{detection} - t_{fault}$ por degradación (primera alarma desde el inicio configurado mientras el fallo está activo; «no detectado» en otro caso) y número de eventos de falsa alarma (flancos de subida sin fallo). No se calculan para métodos que solo reponderan.

18.4 Confianza y pesos

Por sensor: confianza media nominal (antes del primer fallo del sensor), varianza nominal, confianza media y mínima durante el fallo, tiempo de respuesta (desde el inicio hasta $T_i < \text{trust_threshold}$), tiempo de recuperación (desde el fin del fallo hasta $T_i \geq \text{recovery_fraction} \cdot \text{media nominal}$), varianza de la confianza y razón de reducción (media degradada / media nominal). Para pesos: peso medio nominal y global, peso mínimo, porcentaje del peso total asignado durante el fallo, tiempo de reducción ($w_i < \text{weight_fraction} \cdot \text{nominal}$) y tiempo de recuperación.

18.5 Robustez

Criterios explícitos: **divergencia** si la estimación no es finita o $\max \vee e_k$ supera $\text{divergence_threshold}$ (por defecto 1000 veces la mayor desviación nominal); **éxito** si no diverge y $RMSE \leq RMSE_{max}$; **pérdida de desempeño** $RMSE/RMSE_{nominal} - 1$, con $RMSE_{nominal}$ del mismo método sobre el gemelo nominal; **calidad de recuperación** $RMSE_{post}/RMSE_{pre}$. En Monte Carlo se reportan tasas de éxito y de divergencia.

18.6 Estadística

Para múltiples corridas (statistics/): media, mediana, desviación estándar, mínimo, máximo, percentiles 5/25/75/95 e intervalo de confianza de la media $\hat{x} \pm t_{1-\alpha/2, n-1} s/\sqrt{n}$. Las comparaciones entre métodos son **pareadas** (mismos escenarios): diferencia media y mediana, fracción de corridas ganadas, prueba de rangos con signo de Wilcoxon [12], corrección de Holm–Bonferroni para comparaciones múltiples e intervalo bootstrap de la diferencia media; además, distribuciones (diagramas de caja, FDA empírica, histogramas).

19. Arquitectura

19.1 Arquitectura científica

Todo experimento recorre la misma cadena. El simulador genera el valor verdadero del sistema; el generador de sensores virtuales produce las mediciones, y el motor de ruido, incertidumbre y degradación las altera según la configuración. Con esas mediciones se forma el conjunto de datos del escenario, que procesan los algoritmos de fusión y estimación mientras SensorTrust estima la confianza en cada sensor. A partir de la estimación del estado se calculan las métricas, se hace el análisis estadístico y, por último, los resultados se guardan de forma que puedan reproducirse.

19.2 Capas

La lógica científica está completamente separada de la interfaz gráfica. La GUI solo edita la configuración y llama al motor (`ExperimentManager(results).run(config)`, `run_montecarlo`, `run_robustness`, `run_sensitivity`, `run_sweep`) en un hilo de trabajo (`QThread`); el mismo motor se usa desde Python y desde la línea de comandos.

19.3 Idiomas, mensajes y manejo de errores

Internamente, el motor trabaja con identificadores y mensajes en inglés, que son también los que aparecen en la línea de comandos y en los archivos de resultados. La interfaz, en cambio, muestra todo en el idioma que elija el usuario. Los textos se traducen con `i18n.tr`; los nombres de métricas, estados, parámetros, degradaciones y opciones salen de `labels.py`, y los mensajes de error del motor pasan por `i18n_errors.translate_error`, que reconoce cada plantilla de mensaje y conserva sus partes variables (nombres de sensores, valores y límites). Cuando un archivo YAML o JSON tiene un error de sintaxis se indican la línea y la columna, y los errores del sistema de archivos se describen en el idioma de la interfaz. Se verificó que todos los mensajes de error del motor y todos los textos de la interfaz tengan traducción, que todos los parámetros tengan ayuda en los dos idiomas y que cada control visible muestre una explicación emergente.

La aplicación está hecha para no cerrarse ante un error. La configuración se valida antes de ejecutar y los cálculos corren en un hilo aparte que captura las excepciones; si aparece una excepción inesperada, se registra y se informa en un aviso. Antes de cada ejecución se comprueba que se pueda escribir en la carpeta de resultados, y si el usuario cierra la ventana en medio de un cálculo, el programa pide confirmación y lo detiene de forma ordenada. La línea de comandos, por su parte, responde a una entrada no válida con un mensaje de error y el código de salida 2, sin mostrar trazas internas.

19.4 API interna

Tabla 7. Interfaces de la API interna.

Interfaz	Responsabilidad	Ubicación
<code>DynamicModel</code>	f , F , Q , entradas, estados	<code>models/base.py</code>
<code>VirtualSensor / SensorSpec</code>	cadena de medición / especificación nominal	<code>sensors/sensor.py</code>
<code>NoiseModel</code>	realización y varianza del ruido	<code>sensors/noise.py</code>
<code>Degradation / Profile</code>	efecto y perfil temporal	<code>degradation/</code>
<code>GaussianFilter</code>	KF, EKF, UKF	<code>estimators/filters.py</code>
<code>TrustEstimator / Detector</code>	confianza / detección	<code>trust/</code>
<code>FusionMethod / MethodResult</code>	algoritmo de fusión / resultado	<code>fusion/</code>

Interfaz	Responsabilidad	Ubicación
ExperimentManager / ExperimentResult	experimento	experiments/manager.py
evaluate / flatten	métricas	metrics/evaluate.py

Los registros MODEL_REGISTRY, NOISE_REGISTRY, DEGRADATION_REGISTRY, TRUST_REGISTRY y METHOD_PRESETS permiten incorporar nuevos componentes sin modificar los existentes.

20. Módulos

Tabla 8. Paquetes del software y su contenido.

Paquete	Contenido
models	interfaz, seis modelos predefinidos, modelo lineal personalizado, discretización
simulation	generador del valor verdadero, conjunto de datos de escenario, MeasurementSet
sensors	sensores virtuales y ocho modelos de ruido
degradation	quince degradaciones, ocho perfiles, recuperación
estimators	KF, EKF, UKF
trust	momentos de la innovación, estimadores de confianza, detectores
fusion	métodos estáticos, fusión centralizada (casos A–D), fusión a nivel de estimaciones, registro
metrics	estimación, consistencia, detección, confianza, pesos, robustez
statistics	estadística descriptiva y pruebas pareadas
experiments	configuración, gestor, benchmarks, Monte Carlo, diseño, barridos, robustez, sensibilidad
visualization	estilo y figuras científicas
io	exportación/importación de conjuntos de datos, directorios de resultados
gui	interfaz PySide6 bilingüe (sin lógica científica): pestañas, formularios generados desde esquemas de parámetros, ayudas (tooltips.py)
utils	errores, semillas, registro de eventos, álgebra lineal, rutas de parámetros
cli, i18n, i18n_errors, labels, version	línea de comandos, textos bilingües, traducción de los mensajes de error del motor, nombres legibles de métricas, estados, parámetros y opciones, versión

21. Flujo de datos

1. **Configuración** (YAML/JSON o GUI) → `normalize_config` valida tipos, rangos, dimensiones, covarianzas, nombres únicos, métodos y que los instantes de fallo estén dentro del horizonte.
2. **Escenario**: modelo → valor verdadero → sensores (ruido, degradación, disponibilidad) → Scenario con huella SHA-256.
3. **Métodos**: cada método recibe `scenario.measurements()` (sin valor verdadero); los oráculos reciben además `OracleInfo`.
4. **Gemelo nominal** (opcional): mismo escenario sin degradaciones para la pérdida de desempeño.
5. **Métricas** → tabla resumen y métricas completas.

6. **Figuras y almacenamiento** en un directorio con identificador ordenado y manifiesto.

22. Configuración

Así se ve la configuración de un experimento. El fragmento corresponde a `configs/abrupt_bias.yaml`; las claves del archivo están en inglés porque forman parte de la sintaxis que lee el programa.

```
experiment: {name: ST-BENCH-02 - Abrupt bias, seed: 42}
model: {type: constant_velocity, params: {q: 0.05}, x0: [0.0, 1.0], P0: [0.01, 0.01]}
simulation: {duration: 60.0, fs: 100.0}
sensors:
  - {name: S1, measures: position, noise: {type: gaussian, std: 0.1}}
  - name: S2
    measures: position
    noise: {type: gaussian, std: 0.15}
    degradations:
      - {type: bias, severity: 1.0, profile: {type: step, start: 20.0}}
  - {name: S3, measures: position, noise: {type: gaussian, std: 0.2}}
estimation: {x0: [0.0, 1.0], P0: [1.0, 1.0]}
methods: [{name: kf}, {name: adaptive_kf}, {name: sensortrust_kf, trust: {ewma_lambda: 0.98}}]
metrics: {target_state: position, warmup: 1.0, rmse_max: 0.05}
output: {figures: [png, svg], figure_dpi: 150}
```

Secciones adicionales: `montecarlo` (corridas, semilla maestra, diseño, procesos, método de referencia, parámetros variables con ruta, distribución y límites) y `analysis` (robustness, sweep o sensitivity). Los parámetros se referencian con rutas como `sensors[1].degradations[0].severity` o `methods[3].trust.ewma_lambda`. El directorio `configs/` contiene configuraciones con nombre para los 16 benchmarks y los estudios de referencia; `examples/` contiene los 12 ejemplos de la interfaz.

23. Formatos de datos

Tabla 9. Archivos de resultados de un experimento y su contenido.

Archivo	Contenido
<code>config.yaml</code>	configuración normalizada (re-ejecutable)
<code>manifest.json</code>	identificador, software, versión, fecha, modelo, duración, muestreo, sensores, fallos, métodos, semilla, SHA-256 de la configuración y del conjunto de datos, resumen de métricas, entorno, archivos
<code>dataset.csv + dataset_meta.json</code>	<code>time</code> , <code>true_<estado></code> , <code>input_<u></code> , <code><sensor></code> , <code>fault_<sensor></code> , <code>truebias_<sensor></code> , <code>truevar_<sensor></code> , <code>clean_<sensor></code> y metadatos
<code>estimates.csv</code>	estimaciones y desviaciones $\sqrt{P_{jj}}$ por método
<code>trust.csv</code>	confianza, pesos, alarmas, momentos, R^{eff} y $\hat{R}$ por método y sensor
<code>innovations.csv</code>	innovación, su desviación, NIS por sensor y NIS total
<code>metrics.json, metrics.csv</code>	métricas completas y tabla resumen
<code>figures/</code>	PNG, SVG y/o PDF
<code>run.log</code>	registro de la ejecución

Archivo	Contenido
---------	-----------

Los estudios Monte Carlo (ST_MC_...) guardan `design.csv`, `runs.csv`, `summary.csv`, `success_rate.csv` y `paired_tests.csv`; los de robustez (ST_ROB_...), `envelope.csv`, `tolerable_severity.csv` y `map.csv`; los barridos (ST_SWP_...), `aggregate.csv`; los de sensibilidad (ST_SEN_...), `samples.csv` e `indices.csv`. La importación opcional acepta CSV, TXT (delimitador detectado) y JSON (lista de registros o `{"data": [...]}`) con mapeo explícito de columnas de tiempo, sensores, valor verdadero opcional y fallos opcionales; sin valor verdadero se desactivan automáticamente las métricas que lo requieren.

24. Reproducibilidad

La reproducibilidad descansa en varios mecanismos. Todo proceso estocástico usa un generador derivado de la semilla (`utils/rng.py`) y, además, se siembran los generadores globales de `random` y `numpy.random`; la semilla se valida y debe ser un entero en $[0, 2^{63} - 1]$. Cada semilla da lugar a un único escenario, y todos los métodos trabajan sobre esos mismos datos. Los experimentos reciben identificadores consecutivos del tipo ST_EXP_<año>_<n>, que se reservan de forma atómica, y cada uno guarda un manifiesto con las huellas SHA-256 de la configuración normalizada y del conjunto de datos (valor verdadero y mediciones). Con `python -m sensortrust verify results/<ID>` el escenario se vuelve a generar y se comparan ambas huellas. En los estudios de varias corridas, la semilla de cada corrida se deriva de la semilla maestra y se guarda junto con sus parámetros.

El entorno de referencia en el que se obtuvieron los resultados de este documento fue Python 3.11 con NumPy 2.4.6, SciPy 1.17.1, pandas 3.0.6, Matplotlib 3.11.2, PyYAML 6.0.1, SALib 1.6.0 y PySide6 6.11.2. Estas versiones quedan fijadas en `requirements-lock.txt`. Con otras versiones de las bibliotecas los resultados coinciden dentro de la precisión numérica, aunque los últimos decimales pueden variar ligeramente.

25. Verificación del software

Durante el desarrollo se verificó el comportamiento de cada componente frente a resultados conocidos:

Tabla 10. Verificaciones realizadas por componente.

Componente	Verificaciones
Modelos	dimensiones; discretización (Van Loan frente a la fórmula analítica, ZOH frente a <code>expm</code>); estado estacionario térmico; periodo del péndulo; jacobiano; modelos del usuario; errores de configuración
Sensores y ruido	varianza de cada tipo de ruido; soporte del ruido uniforme; autocorrelación AR(1); frecuencia, retardo, resolución y rango; número arbitrario de sensores; los estimadores no acceden al valor verdadero
Degradación	ausencia de efecto antes de t_f para los 15 tipos; valores exactos de sesgo, sesgo progresivo, deriva, varianza, congelamiento, sensibilidad, pérdida y recuperación; formas de perfil; fallos combinados
Filtro de Kalman	ecuación de Riccati; caso escalar analítico; sensor sin ruido; consistencia NIS/NEES; blancura de la innovación; KF = EKF = UKF en modelos lineales; EKF y UKF en el péndulo
Momentos y confianza	estimadores de momentos; segundo momento frente a varianza; límites de la confianza; normalización de pesos; respuesta a sesgo,

Componente	Verificaciones
	varianza y recuperación; sensor dominante (consenso); detección y falsas alarmas
Métricas	valores con resultado conocido; fases del fallo; NEES; detección; confianza y pesos; intervalos de confianza y pruebas pareadas
Reproducibilidad	misma semilla, mismos datos y resultados; mismo conjunto de datos para todos los métodos; gemelo nominal; Monte Carlo en paralelo igual al secuencial
Entrada y salida	estructura del directorio de resultados; comando <code>verify</code> ; configuraciones YAML/JSON válidas e inválidas; exportación e importación; archivos corruptos; métodos no aplicables
Estudios	diseños de muestreo; Monte Carlo; envoltorio de robustez; sensibilidad OAT, Morris y Sobol; barridos
Interfaz gráfica	construcción de la ventana; alternancia español/inglés; carga de todos los ejemplos; todas las figuras para cada método y sensor; entradas no válidas con mensajes traducidos y sin cierres

La reproducibilidad de cualquier experimento guardado puede comprobarse en todo momento con `python -m sensortrust verify results/<ID>` (sección 24).

25.1 Robustez de la interfaz gráfica

El flujo de integración continua del repositorio (GitHub Actions, archivo `.github/workflows/build-windows-exe.yml`) compila el ejecutable de Windows y lo somete a cuatro comprobaciones: apertura de la ventana real, permanencia de la ventana abierta durante 90 s sin cerrarse sola, ejecución de un benchmark por línea de comandos y ejecución de un Monte Carlo con procesos paralelos.

Por diseño, la aplicación no se cierra ante errores inesperados: una excepción no controlada se registra en `sensortrust_gui.log` (carpeta Documentos/SensorTrustFusion) y se informa en un cuadro de aviso, mientras la ventana sigue abierta.

26. Validación científica y resultados de referencia

Los resultados siguientes se obtuvieron ejecutando el software con las configuraciones incluidas (configuración base: modelo de velocidad constante, $q=0,05 \text{ m}^2/\text{s}^3$, 60 s a 100 Hz, tres sensores de posición con $\sigma = 0,10$; 0,15 y 0,20 m; semilla 42; especificación $R M S E_{max}=0,05 \text{ m}$). Cada resultado puede reproducirse con la línea de comandos:

Tabla 11. Comandos para reproducir los resultados de referencia.

Resultado	Comando
Benchmarks ST-BENCH-01 a ST-BENCH-16	<code>python -m sensortrust benchmark all (o benchmark ST-BENCH-02, etc.)</code>
Comparación de métodos	<code>python -m sensortrust run examples/example_10_method_comparison.yaml</code>
Monte Carlo de sesgo abrupto	<code>python -m sensortrust montecarlo configs/montecarlo_abrupt_bias.yaml</code>
Ablación de momentos	<code>python -m sensortrust montecarlo configs/montecarlo_ablation_moments.yaml</code>
Envoltorio y mapa de robustez	<code>python -m sensortrust robustness configs/robustness_abrupt_bias.yaml</code>
Barrido sesgo $\times$ ruido	<code>python -m sensortrust sweep configs/sweep_bias_sigma.yaml</code>
Sensibilidad Morris y Sobol	<code>python -m sensortrust sensitivity configs/sensitivity_morris.yaml y configs/sensitivity_sobol.yaml</code>

Los mismos estudios pueden ejecutarse desde la interfaz gráfica cargando el archivo de configuración correspondiente.

26.1 Caso nominal y consistencia

Tabla 12. Desempeño y consistencia en el caso nominal (ST-BENCH-01).

Método	RMSE [m]	MAE [m]	Error máx. [m]	NIS/m	Fracción NIS en IC95	NEES/n
Promedio simple	0.0898	0.0717	0.3577	–	–	–
Ponderación por inverso de varianza	0.0766	0.0614	0.3174	–	–	–
Filtro de Kalman (R nominal)	0.0206	0.0164	0.0688	1.005	0.949	1.080
KF adaptativo (R Sage-Husa)	0.0207	0.0165	0.0696	0.992	0.953	1.086
KF con compuerta NIS	0.0219	0.0173	0.0729	0.921	0.962	1.145
SensorTrust KF (dos primeros momentos)	0.0207	0.0165	0.0688	1.000	0.950	1.079
KF con R verdadera (oráculo)	0.0206	0.0164	0.0688	1.005	0.949	1.080
KF (solo S1)	0.0252	0.0202	0.0811	1.009	0.950	1.129
KF (solo S2)	0.0337	0.0267	0.1203	1.014	0.947	0.956
KF (solo S3)	0.0447	0.0358	0.1570	0.994	0.948	1.098

En el caso nominal (ST-BENCH-01) los filtros son consistentes (NIS/m y NEES/n cercanos a 1) y la fusión de los tres sensores con KF reduce el RMSE respecto a cualquier sensor individual; SensorTrust KF coincide con el KF nominal, es decir, la adaptación no degrada el desempeño cuando no hay fallos.

26.2 Benchmarks de degradación

La tabla siguiente reúne el RMSE [m] de la posición en los benchmarks ST-BENCH-01 a ST-BENCH-12.

Tabla 13. RMSE de posición [m] en los benchmarks ST-BENCH-01 a ST-BENCH-12.

Benchmark	Promedio simple	KF (R nominal)	KF adaptativo	KF + compuerta NIS	SensorTrust KF	KF R verdadera (oráculo)
ST-BENCH-01 Nominal	0.0898	0.0206	0.0207	0.0219	0.0207	0.0206
ST-BENCH-02 Sesgo abrupto	0.2889	0.2174	0.0241	0.0239	0.0224	0.2174
ST-BENCH-03 Sesgo progresivo	0.3297	0.2510	0.0256	0.0315	0.0227	0.2510
ST-BENCH-04 Varianza creciente	0.2653	0.0536	0.0222	0.0253	0.0223	0.0222
ST-BENCH-05 Deriva	0.1575	0.1045	0.0273	0.0424	0.0225	0.1045
ST-BENCH-06 Valores atípicos	0.1786	0.0341	0.0220	0.0219	0.0221	0.0219
ST-BENCH-07 Sensor congelado	0.8275	0.6473	0.0256	0.0327	0.0233	0.6473
ST-BENCH-08 Pérdida de muestras	0.0941	0.0221	0.0222	0.0234	0.0221	0.0221
ST-BENCH-09 Recuperación	0.2806	0.2105	0.0228	0.0270	0.0222	0.2105
ST-BENCH-10 Dos fallos simultáneos	0.4090	0.2177	0.0262	0.0256	0.0238	0.2533
ST-BENCH-11 Sesgo + ruido	0.2866	0.1681	0.0232	0.0258	0.0224	0.0252
ST-BENCH-12 Múltiples	0.3492	0.2954	0.0334	0.0336	0.0215	0.2954

Benchmark	Promedio simple	KF (R nominal)	KF adaptativo	KF + compuerta NIS	SensorTrust KF	KF R verdadera (oráculo)
perfiles de degradación						

Para esos mismos benchmarks, la tabla siguiente muestra cómo detecta SensorTrust el fallo y cuánto baja la confianza del sensor degradado (detector `moment_test`, $\alpha = 10^{-3}$).

Tabla 14. Detección y confianza del sensor degradado con SensorTrust.

Benchmark	Método	Fallos detectados	Retardo medio [s]	F1 medio	Tasa de falsa alarma	Confianza mínima
ST-BENCH-02	SensorTrust KF (dos primeros momentos)	1/1	0.04	0.989	0.0142	0.010
ST-BENCH-03	SensorTrust KF (dos primeros momentos)	1/1	1.00	0.977	0.0142	0.010
ST-BENCH-04	SensorTrust KF (dos primeros momentos)	1/1	0.90	0.981	0.0141	0.010
ST-BENCH-05	SensorTrust KF (dos primeros momentos)	2/2	5.10	0.909	0.0142	0.010
ST-BENCH-06	SensorTrust KF (dos primeros momentos)	1/1	0.21	0.940	0.0340	0.010
ST-BENCH-07	SensorTrust KF (dos primeros momentos)	1/1	0.13	0.987	0.0141	0.010
ST-BENCH-08	SensorTrust KF (dos primeros momentos)	1/2	26.74	0.065	0.0114	0.891
ST-BENCH-09	SensorTrust KF (dos primeros momentos)	1/1	0.04	0.958	0.0197	0.010
ST-BENCH-10	SensorTrust KF (dos primeros momentos)	2/2	0.04	0.994	0.0270	0.010
ST-BENCH-11	SensorTrust KF (dos primeros momentos)	2/2	0.39	0.986	0.0142	0.010
ST-BENCH-12	SensorTrust KF (dos primeros momentos)	3/3	0.98	0.898	0.0643	0.010
ST-BENCH-13	SensorTrust EKF	1/1	0.05	0.999	0.0056	0.010
ST-BENCH-14	SensorTrust KF (dos primeros momentos)	1/1	5.10	0.978	0.0000	0.010
ST-BENCH-15	SensorTrust KF (dos primeros momentos)	1/1	0.13	0.990	0.0144	0.010
ST-BENCH-16	SensorTrust KF (dos primeros momentos)	1/1	0.99	0.977	0.0219	0.010

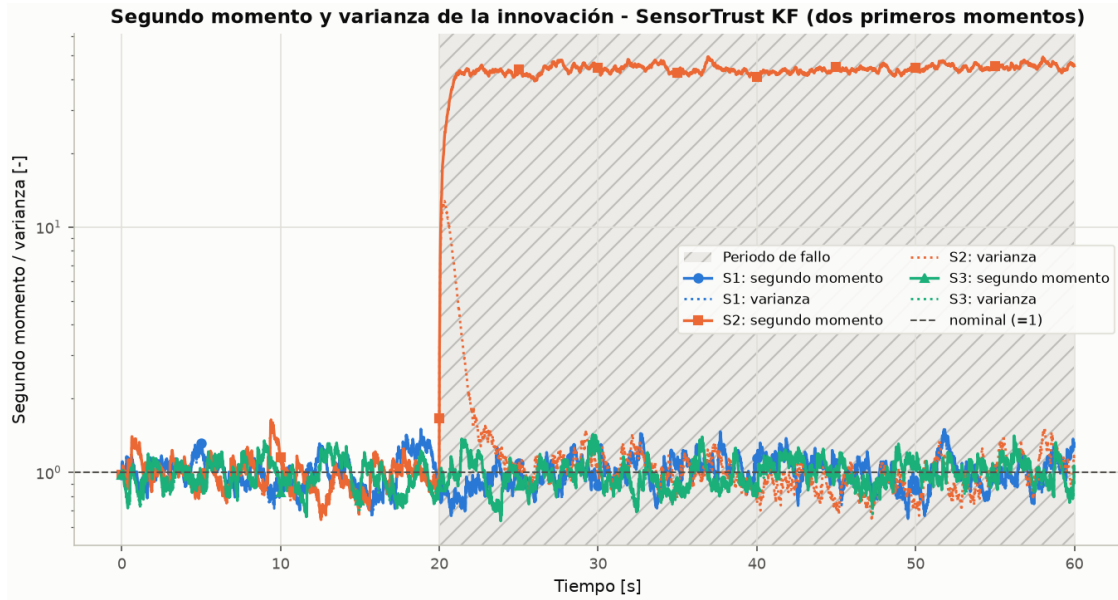


Figura 1. Segundo momento y varianza de la innovación en ST-BENCH-02 (sesgo abrupto en S2): el sesgo eleva el segundo momento pero no la varianza central

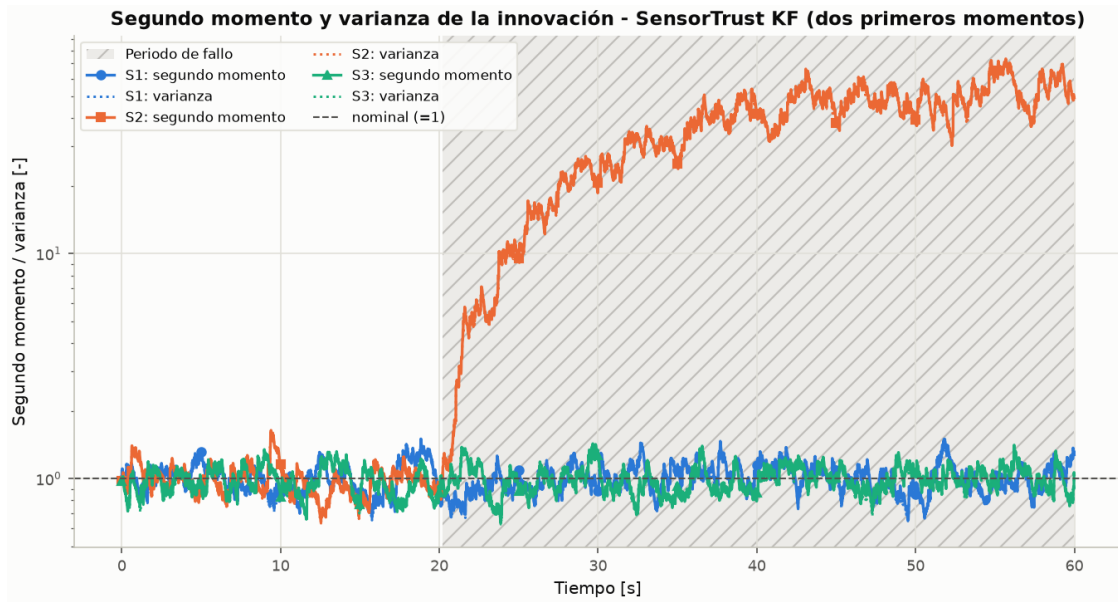


Figura 2. Segundo momento y varianza de la innovación en ST-BENCH-04 (varianza creciente en S2): ambos aumentan

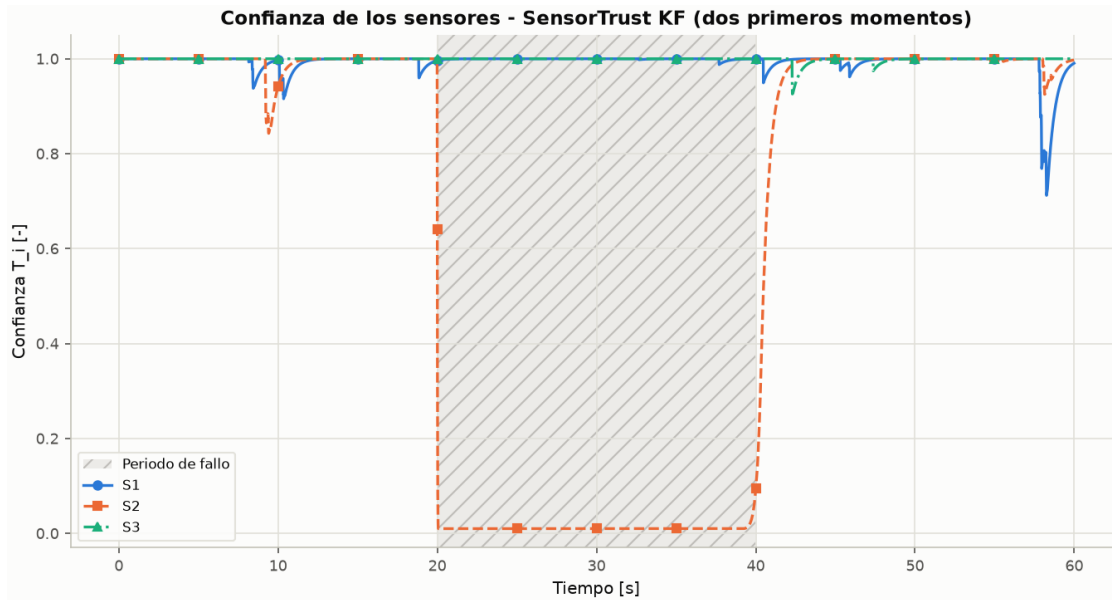


Figura 3. Confiabilidad en ST-BENCH-09 (sesgo con recuperación en S2): disminución rápida y recuperación posterior

De estos resultados se desprenden varias observaciones. El KF con R nominal y los promedios estáticos incumplen la especificación cuando hay sesgos, deriva o congelamiento. El KF con la R verdadera (oráculo) corrige el aumento de varianza pero no el sesgo, porque el sesgo no es una propiedad de R . SensorTrust KF se mantiene cerca del RMSE nominal en todos los casos en que el fallo puede detectarse a través de la innovación. La pérdida de muestras (ST-BENCH-08), en cambio, no se detecta con la innovación, aunque tampoco afecta de forma apreciable al KF.

26.3 Otros modelos

Tabla 15. Resultados en otros modelos dinámicos y con sensor dominante degradado (ST-BENCH-13 a ST-BENCH-16).

Benchmark	Método	RMSE	RMSE durante fallo	NEES/n
ST-BENCH-13	EKF (R nominal)	0.0249	0.0309	30.740
ST-BENCH-13	UKF (R nominal)	0.0249	0.0309	30.740
ST-BENCH-13	EKF adaptativo (R Sage-Husa)	0.0050	0.0053	1.067
ST-BENCH-13	SensorTrust EKF	0.0048	0.0050	1.013
ST-BENCH-13	SensorTrust UKF	0.0048	0.0050	1.013
ST-BENCH-14	Promedio simple	0.3885	0.4649	—
ST-BENCH-14	Filtro de Kalman (R nominal)	0.3002	0.3611	26.577
ST-BENCH-14	KF adaptativo (R Sage-Husa)	0.0514	0.0526	1.161
ST-BENCH-14	SensorTrust KF (dos primeros momentos)	0.0481	0.0480	1.070
ST-BENCH-15	Filtro de Kalman (R nominal)	0.1288	0.1564	1119.708
ST-BENCH-15	KF adaptativo (R Sage-Husa)	0.0039	0.0041	1.218
ST-BENCH-15	SensorTrust KF (dos primeros momentos)	0.0034	0.0035	1.050
ST-BENCH-15	Fusión de estimaciones CI (pesos iguales)	0.1219	0.1481	284.452
ST-BENCH-15	SensorTrust fusión de estimaciones CI	0.0051	0.0057	0.729
ST-BENCH-16	Filtro de Kalman (R nominal)	0.3463	0.3964	633.831
ST-BENCH-16	KF adaptativo (R Sage-Husa)	0.4462	0.5109	842.024

Benchmark	Método	RMSE	RMSE durante fallo	NEES/n
ST-BENCH-16	KF con compuerta NIS	0.4445	0.5089	863.747
ST-BENCH-16	SensorTrust KF (dos primeros momentos)	0.0215	0.0234	1.155
ST-BENCH-16	SensorTrust KF (sin consenso, ablación)	0.4531	0.5188	884.776

26.4 Sensor dominante (ST-BENCH-16)

Cuando el sensor más preciso (S2, 0,05 m) desarrolla un sesgo progresivo, los métodos que evalúan cada innovación de forma aislada atribuyen el fallo a los sensores sanos. El RMSE de SensorTrust KF es 0.0215 m frente a 0.4531 m sin la referencia de consenso, 0.3463 m del KF nominal y 0.4462 m del KF adaptativo.

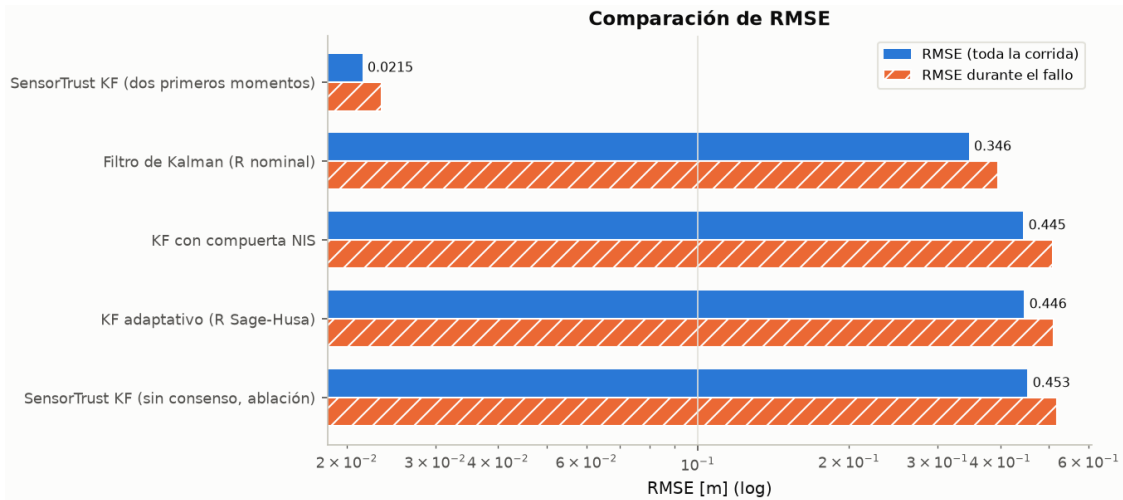


Figura 4. Comparación de RMSE en ST-BENCH-16

26.5 Comparación de métodos (sesgo + ruido)

Tabla 16. Comparación de métodos bajo sesgo y ruido sobre el mismo conjunto de datos.

Método	RMSE [m]	RMSE fallo [m]	NIS/m	NEES/n	Tiempo [s]
Promedio simple	0.2866	0.3429	–	–	0.00
Ponderación por inverso de varianza	0.2282	0.2724	–	–	0.00
Filtro de Kalman (R nominal)	0.1681	0.2038	9.363	63.102	0.59
KF adaptativo (R Sage-Husa)	0.0232	0.0242	1.014	1.169	1.13
KF con compuerta NIS	0.0258	0.0275	0.957	1.367	0.64
SensorTrust KF (solo primer momento)	0.0225	0.0234	0.915	1.092	1.22
SensorTrust KF (solo segundo momento)	0.0224	0.0232	0.906	1.092	1.33
SensorTrust KF (dos primeros momentos)	0.0224	0.0232	0.903	1.094	1.26
Fusión de estimaciones CI (pesos iguales)	0.1699	0.2059	–	31.831	1.25
SensorTrust fusión de estimaciones CI	0.0233	0.0238	–	0.795	1.89
KF con R verdadera (oráculo)	0.0252	0.0272	1.292	1.358	0.67
Mejor sensor (oráculo)	0.1005	0.1005	–	–	0.00

26.6 Monte Carlo

Estudio montecarlo_abrupt_bias: 100 corridas con $t_f \sim U(10,30)$ s, $b \sim U(0,5,3)$ m y $\sigma_{s2} \sim U(0,05,0,5)$ m.

Tabla 17. Resumen estadístico del estudio Monte Carlo de sesgo abrupto.

Método	Métrica	n	Media	Mediana	Desv.	P95	IC95 inf.	IC95 sup.
Ponderación por inverso de varianza	rmse	100	0.2273	0.1329	0.2160	0.7388	0.1844	0.2701
Ponderación por inverso de varianza	rmse_during_fault	100	0.2908	0.1776	0.2762	1.0054	0.2360	0.3456
Filtro de Kalman (R nominal)	rmse	100	0.2006	0.1056	0.2284	0.7376	0.1553	0.2459
Filtro de Kalman (R nominal)	rmse_during_fault	100	0.2698	0.1580	0.2872	1.0050	0.2129	0.3268
KF adaptativo (R Sage-Husa)	rmse	100	0.0236	0.0234	0.0014	0.0258	0.0233	0.0239
KF adaptativo (R Sage-Husa)	rmse_during_fault	100	0.0247	0.0243	0.0027	0.0285	0.0242	0.0253
KF con compuerta NIS	rmse	100	0.0242	0.0240	0.0021	0.0273	0.0238	0.0246
KF con compuerta NIS	rmse_during_fault	100	0.0253	0.0246	0.0038	0.0310	0.0246	0.0261
SensorTrust KF (dos primeros momentos)	rmse	100	0.0229	0.0229	0.0011	0.0247	0.0227	0.0231
SensorTrust KF (dos primeros momentos)	rmse_during_fault	100	0.0235	0.0235	0.0018	0.0263	0.0231	0.0238
SensorTrust KF (dos primeros momentos)	mean_detection_delay	100	0.0553	0.0479	0.0244	0.1166	0.0505	0.0601

Tabla 18. Tasa de éxito por método en el estudio Monte Carlo de sesgo abrupto.

Método	Tasa de éxito (RMSE <= 0,05 m)	Tasa de divergencia	Corridas
Ponderación por inverso de varianza	0.00	0.00	100
Filtro de Kalman (R nominal)	0.14	0.00	100
KF adaptativo (R Sage-Husa)	1.00	0.00	100
KF con compuerta NIS	1.00	0.00	100
SensorTrust KF (dos primeros momentos)	1.00	0.00	100

Pruebas pareadas respecto a SensorTrust KF (diferencia = referencia – método; valores negativos favorecen a SensorTrust):

Tabla 19. Pruebas pareadas respecto a SensorTrust KF en el estudio Monte Carlo de sesgo abrupto.

Método comparado	Métrica	n	Mediana de la diferencia (ref. - método)	Fracción de corridas ganadas por la referencia	p (Wilcoxon, Holm)
KF adaptativo (R Sage-Husa)	rmse	100	-0.0005	0.93	5.31e-16
Ponderación por inverso de varianza	rmse	100	-0.1110	1.00	1.56e-17
KF con compuerta NIS	rmse	100	-0.0009	0.97	1.60e-17
Filtro de Kalman (R)	rmse	100	-0.0835	1.00	1.56e-17

Método comparado	Métrica	n	Mediana de la diferencia (ref. - método)	Fracción de corridas ganadas por la referencia	p (Wilcoxon, Holm)
nominal)					
KF adaptativo (R Sage-Husa)	rmse_during_fault	100	-0.0009	0.92	1.38e-15
Ponderación por inverso de varianza	rmse_during_fault	100	-0.1536	1.00	1.56e-17
KF con compuerta NIS	rmse_during_fault	100	-0.0010	0.90	9.76e-16
Filtro de Kalman (R nominal)	rmse_during_fault	100	-0.1348	1.00	1.56e-17

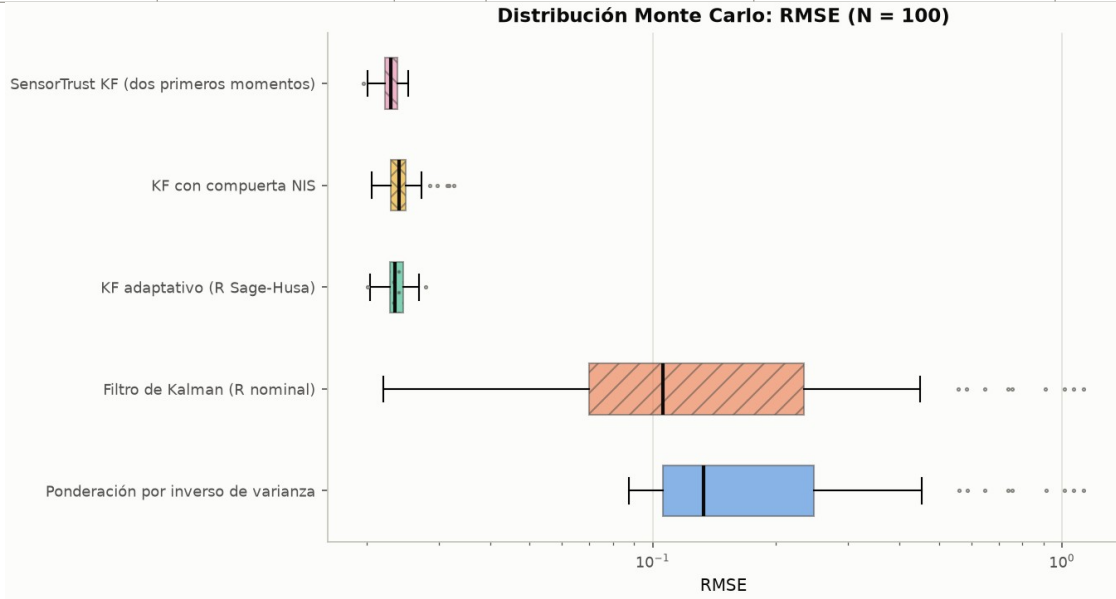


Figura 5. Distribución Monte Carlo del RMSE

Estudio de ablación `montecarlo_ablation_moments` (100 corridas LHS de sesgo $b \sim U(0,1,5)$ más factor de varianza $F \sim U(1,40)$):

Tabla 20. Resumen del estudio de ablación de momentos.

Método	Métrica	n	Media	Mediana	Desv.	P95	IC95 inf.	IC95 sup.
Filtro de Kalman (R nominal)	rmse	100	0.1355	0.1356	0.0693	0.2467	0.1218	0.1493
Filtro de Kalman (R nominal)	rmse_during_fault	100	0.1879	0.1885	0.0978	0.3443	0.1685	0.2073
KF adaptativo (R Sage-Husa)	rmse	100	0.0230	0.0229	0.0014	0.0256	0.0228	0.0233
KF adaptativo (R Sage-Husa)	rmse_during_fault	100	0.0249	0.0247	0.0023	0.0290	0.0245	0.0254
SensorTrust KF (solo primer momento)	rmse	100	0.0228	0.0226	0.0014	0.0246	0.0225	0.0231
SensorTrust KF (solo primer momento)	rmse_during_fault	100	0.0246	0.0242	0.0024	0.0280	0.0241	0.0250
SensorTrust KF (solo primer momento)	mean_detection_delay	100	1.0228	0.7950	0.8997	2.0255	0.8443	1.2013
SensorTrust KF (solo segundo momento)	rmse	100	0.0230	0.0223	0.0032	0.0264	0.0223	0.0236

Método	Métrica	n	Media	Mediana	Desv.	P95	IC95 inf.	IC95 sup.
SensorTrust KF (solo segundo momento)	rmse_during_fault	100	0.0248	0.0238	0.0051	0.0303	0.0238	0.0258
SensorTrust KF (solo segundo momento)	mean_detection_delay	100	0.6637	0.4900	0.6090	1.4930	0.5429	0.7845
SensorTrust KF (dos primeros momentos)	rmse	100	0.0223	0.0222	0.0011	0.0242	0.0220	0.0225
SensorTrust KF (dos primeros momentos)	rmse_during_fault	100	0.0236	0.0235	0.0018	0.0266	0.0232	0.0239
SensorTrust KF (dos primeros momentos)	mean_detection_delay	100	0.5281	0.4750	0.3063	0.8380	0.4673	0.5889

En este estudio, dominado por combinaciones de sesgo y ruido de magnitud moderada, las tres variantes de SensorTrust cumplen la especificación en todas las corridas; la variante de dos momentos obtiene la menor media y mediana del RMSE, con diferencias pequeñas respecto a las variantes de un solo momento. La contribución específica del segundo momento se aprecia con claridad en los casos de aumento puro de varianza (ST-BENCH-04), donde la media de la innovación permanece en cero.

26.7 Envoltente de robustez

Tabla 21. Severidad tolerable del sesgo por método (envoltente de robustez).

Método	Sesgo tolerable b^* [m]	Límite alcanzado en el rango probado
KF adaptativo (R Sage-Husa)	5.000	no (todos los valores cumplen)
Ponderación por inverso de varianza	ninguna	sí
Filtro de Kalman (R nominal)	0.242	sí
SensorTrust KF (dos primeros momentos)	5.000	no (todos los valores cumplen)

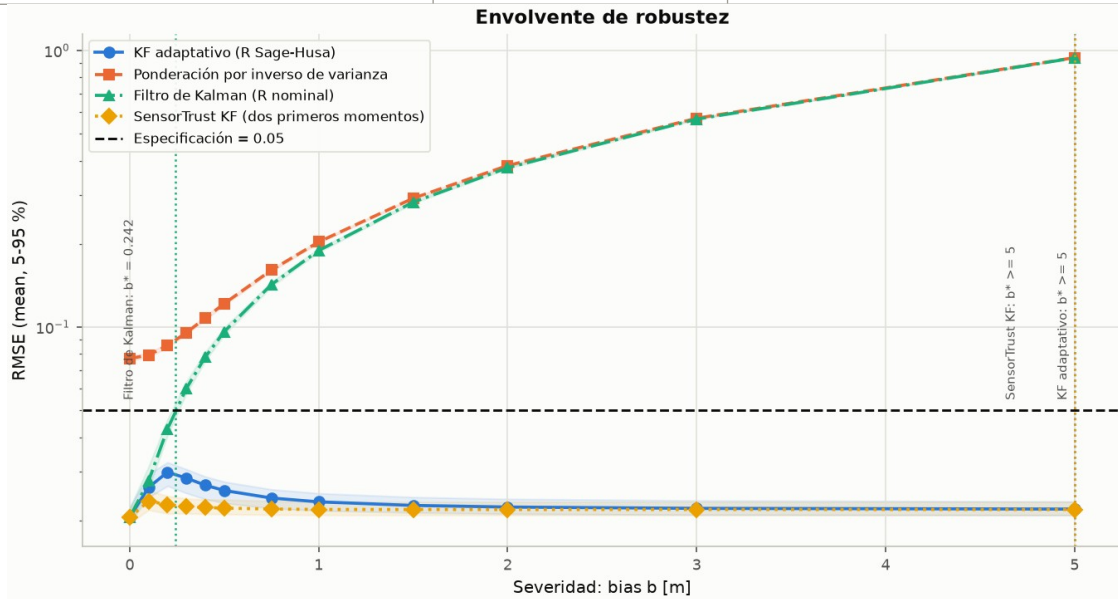


Figura 6. Envoltente de robustez frente al sesgo abrupto de S_2

26.8 Barrido paramétrico

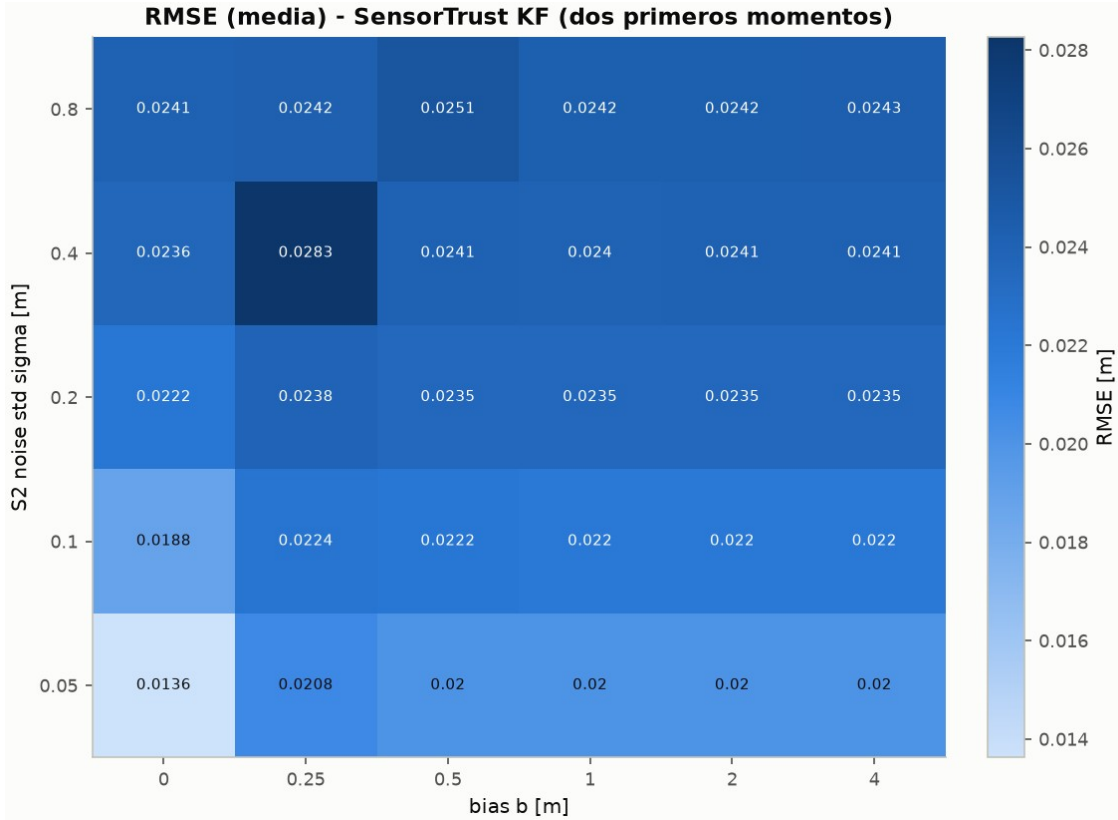


Figura 7. RMSE medio de SensorTrust KF en función del sesgo y de la desviación del ruido de S2

26.9 Sensibilidad

Se estudió qué parámetros explican el RMSE de SensorTrust KF bajo sesgo abrupto: magnitud del sesgo $b \in [0, 1; 3]$ m, desviación del ruido de S2 $\in [0, 05; 0, 4]$ m, instante del fallo $\in [5; 25]$ s, $\lambda \in [0, 9; 0, 995]$, $\gamma_1 \in [1; 5]$ y $\beta_1 \in [0, 1; 2]$ (semilla común). Índices de Morris de $\log_{10}(RMSE)$ (20 trayectorias; parámetro más influyente: S2 noise std [m]; segundo: bias b [m]):

Tabla 22. Índices de sensibilidad de Morris.

Parámetro	mu	mu_star	sigma	mu_star_conf
S2 noise std [m]	-0.0515	0.1550	0.2388	0.0772
bias b [m]	-0.1107	0.1122	0.1514	0.0631
trust scale beta1	0.0309	0.0501	0.0944	0.0362
fault onset [s]	-0.0337	0.0337	0.0395	0.0192
EWMA lambda	-0.0129	0.0130	0.0279	0.0119
dead zone gamma1	0.0036	0.0054	0.0135	0.0056

Índices de Sobol de $\log_{10}(RMSE)$ (N = 256, 2048 evaluaciones; la transformación logarítmica evita que unos pocos valores grandes dominen la descomposición de la varianza; parámetro con mayor índice total: bias b [m]):

Tabla 23. Índices de sensibilidad de Sobol.

Parámetro	S1	S1_conf	ST	ST_conf
bias b [m]	0.3990	0.1943	0.9183	0.5974

Parámetro	S1	S1_conf	ST	ST_conf
S2 noise std [m]	0.0704	0.1415	0.6624	0.4120
trust scale beta1	0.1780	0.1619	0.4290	0.2957
fault onset [s]	0.0596	0.0441	0.0879	0.0375
EWMA lambda	0.0067	0.0163	0.0603	0.0847
dead zone gamma1	-0.0041	0.0401	0.0245	0.0100

Durante una primera ejecución de este estudio, una muestra con $\beta_1 \approx 0,1$ (confianza extremadamente agresiva) produjo un RMSE de 0,24 m: el ruido redujo momentáneamente la confianza de los sensores sanos justo en el instante del fallo y el sensor defectuoso quedó como única referencia del consenso. El análisis de sensibilidad permitió identificar y corregir este mecanismo (ventana de memoria del consenso, capítulo 10.1). Todos los resultados de este documento se obtuvieron con la versión corregida. Con la versión final, 7 de las 2048 evaluaciones de Sobol superan la especificación de 0,05 m (máximo 0.082 m); corresponden a fallos marginales (sesgos del orden de la desviación del ruido de S2) combinados con memoria corta ($\lambda \approx 0,90$) y escalas de confianza agresivas, en los que la confianza puede converger a un equilibrio incorrecto (capítulo 29). Morris y Sobol identifican el mismo conjunto de tres parámetros más influyentes (magnitud del sesgo, desviación del ruido de S2 y escala de confianza β_1), con el orden de los dos primeros intercambiado; los intervalos de confianza de los índices de Sobol son amplios con $N = 256$, por lo que su orden relativo fino no debe sobreinterpretarse. Los parámetros λ y γ_1 tienen efectos pequeños en los rangos estudiados.

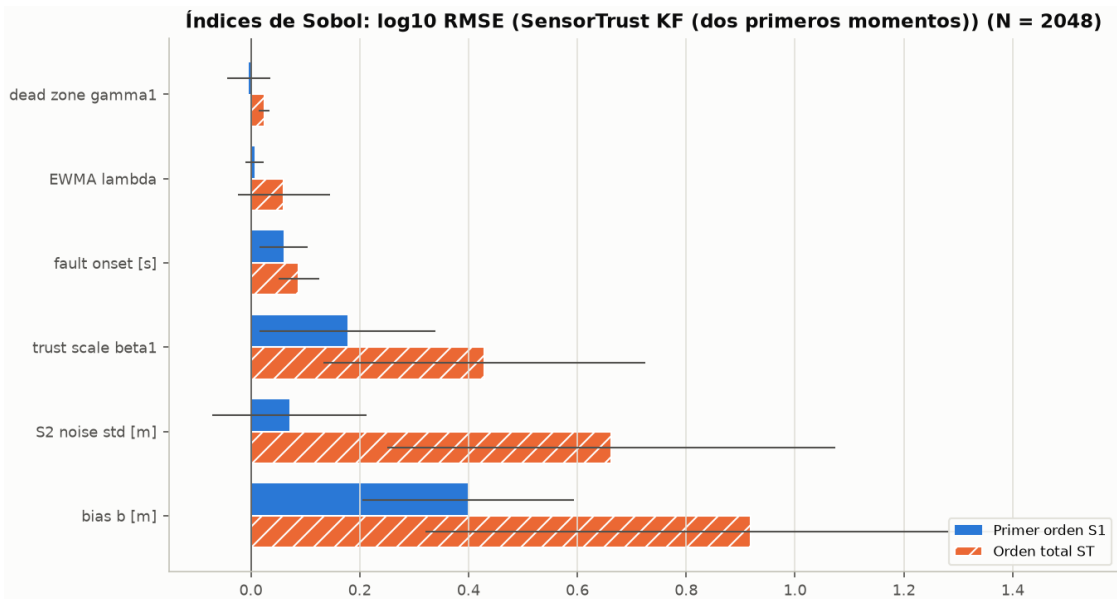


Figura 8. Índices de Sobol de primer orden y totales

26.10 Revisión final

Antes de publicar la versión 1.0.0 se hizo una revisión de conjunto. Se revisaron las dimensiones de matrices y vectores (que se validan en `normalize_config`, `VirtualSensor` y los filtros), la correspondencia entre las ecuaciones del código y las de este documento, los divisores, intervalos y pruebas de la parte estadística, las normalizaciones (innovación estandarizada con la R nominal y pesos que suman 1), las semillas, las definiciones y fases de las métricas, la implementación del filtro (Riccati, forma de Joseph y consistencia), la interpretación de los momentos (segundo momento, varianza, RMS y energía no son lo mismo), los pesos, los escenarios y los resultados.

Las correcciones derivadas de esta revisión (compuerta de innovación, referencia de consenso, prueba de dispersión chi-cuadrado) están documentadas en los capítulos 10, 12 y 15.

27. Instalación

Requisitos: Python 3.10 o superior (probado con 3.11), sistema Windows, Linux o macOS. En un entorno limpio:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
pip install -e .
```

En Linux o macOS, la segunda línea se reemplaza por `source .venv/bin/activate`. El archivo `requirements.txt` indica las versiones mínimas de las dependencias; `requirements-lock.txt` fija las versiones exactas del entorno de referencia (sección 24) y permite reproducirlo con `pip install -r requirements-lock.txt`.

Dependencias: NumPy, SciPy, pandas, Matplotlib, PyYAML, SALib y PySide6. Alternativamente, en Windows puede usarse el ejecutable `SensorTrustFusion-1.0.0-win64.exe` publicado en la sección *Releases* del repositorio, que no requiere instalar Python. El ejecutable se compila con PyInstaller (especificación `packaging/SensorTrustFusion.spec`) en GitHub Actions a partir del código fuente del repositorio; el flujo de compilación está en `.github/workflows/build-windows-exe.yml`, dentro del repositorio. En Linux sin entorno gráfico completo, Qt puede requerir las bibliotecas del sistema `libegl1`, `libgl1`, `libxkbcommon0` y `libfontconfig1`.

28. Ejecución

Interfaz gráfica (en inglés o en español):

```
python -m sensortrust
python -m sensortrust gui --lang es
```

Línea de comandos:

```
python -m sensortrust run configs/abrupt_bias.yaml
python -m sensortrust benchmark all
python -m sensortrust montecarlo configs/montecarlo_abrupt_bias.yaml
python -m sensortrust robustness configs/robustness_abrupt_bias.yaml
python -m sensortrust sweep configs/sweep_bias_sigma.yaml
python -m sensortrust sensitivity configs/sensitivity_morris.yaml
python -m sensortrust generate-dataset configs/nominal.yaml -o datasets
python -m sensortrust import-dataset config.yaml datos.csv --sensor
S1=c1 --sensor S2=c2
python -m sensortrust verify results/ST_EXP_2026_000001
python -m sensortrust list
```

Uso desde Python:

```
from sensortrust.experiments import ExperimentManager, get_benchmark
result = ExperimentManager("results").run(get_benchmark("ST-BENCH-02"))
print(result.table[["method", "rmse", "mean_detection_delay"]])
```

Todas las funciones principales operan sin conexión a Internet una vez instaladas las dependencias.

29. Limitaciones

1. La función que transforma los excesos de los momentos en confianza (exponencial con zonas muertas estadísticas) es una elección de diseño; otras funciones de mapeo son posibles y no se compararon sistemáticamente en esta versión.
2. Los fallos que no alteran la innovación (pérdida de muestras, sesgos comunes a todos los sensores de un grupo) no son identificables por la confianza basada en innovación; la referencia de consenso requiere al menos tres sensores redundantes y una mayoría sana, y con dos sensores un desacuerdo no puede atribuirse.
3. Para fallos marginales (magnitud del orden de la desviación del ruido del sensor) combinados con memoria corta del estimador de momentos ($\lambda \leq 0,9$) y escalas de confianza agresivas, la dinámica de confianza puede converger a un equilibrio incorrecto en el que un sensor sano pierde confianza (observado en pocas evaluaciones del análisis de Sobol, sección 26.9).
4. Ante aumentos de varianza muy grandes, la varianza del estimador de la media puede superar la zona muerta y reducir también el término del primer momento (efecto documentado en las pruebas).
5. Los parámetros por defecto del estimador ($\lambda, \gamma, \beta, \eta$) no se optimizaron; el análisis de sensibilidad permite estudiarlos.
6. La fusión a nivel de estimaciones usa Intersección de Covarianzas (conservadora); no se implementan arquitecturas distribuidas con comunicación.
7. El EKF usa jacobianos por diferencias finitas en el péndulo; el UKF usa parámetros fijos por defecto.
8. El retardo de los sensores se simula pero los estimadores no lo compensan.
9. Las métricas de detección por muestra dependen de la convención de etiquetado (episodio de fallo); en fallos intermitentes la histéresis del detector genera falsos positivos en los intervalos apagados.
10. La ejecución es en tiempo de simulación (no en tiempo real) y la velocidad depende del número de sensores y métodos (del orden de 1 s por método para 6000 muestras y tres sensores en un computador convencional).

Glosario

Innovación: diferencia entre la medición y su predicción. **NIS / NEES:** errores cuadráticos normalizados de innovación y de estimación. **Oráculo:** método de referencia que usa información verdadera no disponible en la práctica. **Gemelo nominal:** mismo escenario sin degradaciones. **Semilla:** entero que determina todas las realizaciones aleatorias. **Huella:** SHA-256 de los datos de un escenario. **Benchmark (escenario de referencia):** experimento con configuración y semilla fijas que sirve para comparar métodos.

Referencias

- [1] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [2] Y. Bar-Shalom, X. R. Li y T. Kirubarajan, *Estimation with Applications to Tracking and Navigation*. Wiley, 2001.

- [3] C. F. Van Loan, "Computing integrals involving the matrix exponential," *IEEE Transactions on Automatic Control*, vol. 23, no. 3, pp. 395–404, 1978.
- [4] S. J. Julier y J. K. Uhlmann, "A non-divergent estimation algorithm in the presence of unknown correlations," *Proc. American Control Conference*, 1997.
- [5] S. J. Julier y J. K. Uhlmann, "Unscented filtering and nonlinear estimation," *Proceedings of the IEEE*, vol. 92, no. 3, pp. 401–422, 2004.
- [6] A. P. Sage y G. W. Husa, "Adaptive filtering with unknown prior statistics," *Joint Automatic Control Conference*, 1969.
- [7] R. K. Mehra, "On the identification of variances and adaptive Kalman filtering," *IEEE Transactions on Automatic Control*, vol. 15, no. 2, pp. 175–184, 1970.
- [8] M. D. McKay, R. J. Beckman y W. J. Conover, "A comparison of three methods for selecting values of input variables in the analysis of output from a computer code," *Technometrics*, vol. 21, no. 2, pp. 239–245, 1979.
- [9] M. D. Morris, "Factorial sampling plans for preliminary computational experiments," *Technometrics*, vol. 33, no. 2, pp. 161–174, 1991.
- [10] J. Herman y W. Usher, "SALib: An open-source Python library for sensitivity analysis," *Journal of Open Source Software*, vol. 2, no. 9, 2017.
- [11] A. Saltelli et al., *Global Sensitivity Analysis: The Primer*. Wiley, 2008.
- [12] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945; S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.